

Chapter 35

The Math Under the Models

Treat each row of a dataset as a vector and measure distance and similarity between rows · multiply a matrix of features by a vector of weights to predict every row at once · read a derivative and a gradient as "which way is downhill, and how steep"

Approved chapter · v1

Contents

Chapter 35. The Math Under the Models

Why this matters

In plain English

35.1 Vectors: every customer is a point

35.2 The dot product: a number for "how aligned"

35.3 Matrices: predicting every row at once

35.4 Loss and the derivative: which way is downhill

35.5 Gradients: the slope in every direction

35.6 Gradient descent, by hand and in NumPy

35.7 Probability distributions: describing what's likely

35.8 Likelihood and maximum likelihood

35.9 Entropy, cross-entropy, and log loss

35.10 Principal component analysis (PCA)

35.11 Which math sits under which model

Common mistakes and how to spot them

In the real world: Meera and the 91% lead score

Tools

The project: gradient descent and PCA from scratch

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 35. The Math Under the Models

Part IV — Machine Learning & Data Science

Chapter at a glance

You will learn to: treat each row of a dataset as a vector and measure distance and similarity between rows · multiply a matrix of features by a vector of weights to predict every row at once · read a derivative and a gradient as "which way is downhill, and how steep" · run gradient descent by hand for three steps, then in NumPy, and spot a bad learning rate or unscaled feature from the loss alone · describe data with the Bernoulli, binomial, Poisson, and normal distributions, and say when a distribution doesn't fit · find a maximum likelihood estimate and connect it to the loss a classifier minimizes · calculate entropy, cross-entropy, and log loss, and use them to judge a probability score · work through principal component analysis (PCA) on a small example by hand, then on all Riverstone customers.

Before you start: Chapter 4 (percentages, averages, compounding), Chapter 13 (the one-year database and the `sales_lines` view), Chapter 17 (running Python code, lists, loops), and Chapter 21 (mean, standard deviation, and basic probability). No calculus or linear algebra is assumed.

Time needed: 10–14 hours of reading and practice, spread over two weeks. Do the hand calculations with a pen before you run the code.

Tools: Python 3 with NumPy and pandas (Chapter 17 covers installation); scikit-learn for two cross-checks. A calculator or spreadsheet for the hand work.

Practice data: three small files built from the one-year Riverstone database (`riverstone_2025`) by `companion/ch35/make_ch35_data.py` : `customers_2025.csv` , `orders_2025.csv` , and `leads_2025.csv` . Every number in this chapter was calculated, and every output shown is real.

Why this matters

You can train a machine learning model in three lines of Python. The trouble starts when it misbehaves, and it will:

- The model's error stops falling after a few rounds of training, or shoots up to a number with 80 digits.
- A colleague's clustering puts every customer in the same group, because one column is measured in rupees and the rest in counts.
- A vendor tells your sales head their lead score is "91% accurate", and nobody in the room knows whether that is good.
- Two models have the same accuracy, and someone asks which to trust.
- An interviewer asks, "Why do we use log loss for logistic regression and not squared error?"

Each of these is a math question in disguise. You don't need proofs to answer them. You need six ideas, understood well enough to calculate by hand: **vectors**, **matrices**, **gradients**, **probability distributions**, **likelihood**, and **entropy**. This chapter teaches those six with Riverstone's numbers, one worked calculation at a time. After it, every algorithm in Chapters 36 to 43 will look like a variation on one theme, and you'll be able to reason about a model instead of guessing.

In plain English

Think of a tailor who makes uniforms for Riverstone's customers.

The tailor writes each customer's measurements on a card: chest, waist, sleeve, height. That list of numbers, always in the same order, is a **vector**. A drawer full of cards, one per customer, is a **matrix**. To compare two customers, the tailor lines their cards up side by side and sees how closely the measurements agree; that comparison is a **dot product**.

The tailor cuts a pattern, the customer tries it on, and the tailor notes how badly it fits: one number for "how wrong". That number is the **loss**. The tailor doesn't start over each time. They ask, *"If I let out the waist a little, does the fit get better or worse, and how quickly?"* That question is the **derivative**, and asking it for every seam at once gives the **gradient**. Adjusting every seam a small amount in the better direction, trying it on again, and repeating is **gradient descent**. Adjust too boldly and the jacket swings from too tight to too loose and never fits; that's a **learning rate** set too high.

When a new customer walks in and the tailor has no card yet, they guess the size from experience of who usually walks in. That's a **probability distribution**. Choosing the guess that makes the customers they've already seen least surprising is **maximum likelihood**, and the tailor's surprise when a guess is wrong is **cross-entropy**.

Finally, the tailor notices that chest, waist, and sleeve mostly move together, so a single "size" (S, M, L) captures most of what the card says. Finding that one summary direction is **principal component analysis**.

Vectors, matrices, gradients, distributions, likelihood, entropy, PCA: that's the whole chapter.

35.1 Vectors: every customer is a point

A row is a list of numbers

"How do I describe a customer so a computer can compare them with another?" Pick some measurements and write them in a fixed order. Here are two Riverstone customers from 2025, described by the number of orders they placed and their net revenue in ₹ lakh (₹100,000):

Customer	Orders	Revenue (₹ lakh)
Sharma Hardware	16	5.03
Metro Mart	16	3.29

As vectors: **Sharma** = [16, 5.03] and **Metro** = [16, 3.29]. A **vector** is an ordered list of numbers. Each number is a **component**, and the number of components is the vector's **dimension**. These are 2-dimensional vectors, so you can draw each customer as a point on a chart with orders across and revenue up. A customer described by seven measurements is a point in 7-dimensional space. You can't draw that, but the arithmetic is the same.

In machine learning, each component is usually called a **feature**. A model never sees "Sharma Hardware"; it sees the vector.

Distance between two customers

"How different are Sharma Hardware and Metro Mart?" Treat them as points and measure the straight-line distance, the **Euclidean distance**. Subtract component by component, square, add, and take the square root:

$$\text{distance} = \sqrt{((16 - 16)^2 + (5.03 - 3.29)^2)} = \sqrt{(0 + 1.74^2)} = \sqrt{3.0276} = 1.74$$

The **length** (or **norm**) of a single vector is its distance from zero. For Sharma: $\sqrt{(16^2 + 5.03^2)} = \sqrt{(256 + 25.30)} = \sqrt{281.30} = 16.77$.

Here is the same arithmetic in NumPy, the Python library for working with arrays of numbers:

```
import numpy as np
import pandas as pd

sharma = np.array([16, 5.03])
metro = np.array([16, 3.29])

difference = sharma - metro
distance = np.sqrt(np.sum(difference ** 2))
print("difference:", difference)
print("distance: ", round(distance, 2))
print("same with np.linalg.norm:", round(np.linalg.norm(sharma - metro), 2))
print("length of sharma:", round(np.linalg.norm(sharma), 2))
```

```
difference: [0.  1.74]
distance:   1.74
same with np.linalg.norm: 1.74
length of sharma: 16.77
```

How it works:

- `np.array([16, 5.03])` makes a vector. Arithmetic on arrays happens **element by element**: `sharma - metro` subtracts 16 from 16 and 3.29 from 5.03 in one step, with no loop.
- `difference ** 2` squares each component; `np.sum` adds them; `np.sqrt` takes the root. That's the formula, line for line.
- `np.linalg.norm` does the same calculation in one call. Use it in real work, but know what it does.

Watch out: units decide distance. Measure revenue in rupees instead of lakh and Sharma becomes [16, 502,775]. Now a difference of 1 order counts as much as a difference of ₹1, so distance is driven almost entirely by revenue, and orders might as well not exist. Any method built on distance (k-nearest neighbors, k-means clustering, PCA) is affected. The fix is **scaling**: put every feature on a comparable scale before measuring distance. Section 35.10 shows the damage with real numbers, and Chapter 36 turns scaling into a standard step.

The whole customer table as vectors

Load the practice file and look at a few customers. Each row is a vector of features:

```
customers = pd.read_csv("customers_2025.csv")
print(len(customers), "customers who ordered in 2025")
view = customers[["customer_name", "orders", "revenue", "storage_share",
                  "kitchen_share", "industrial_share", "furniture_share"]]
view.columns = ["customer", "orders", "revenue", "storage", "kitchen", "industrial", "furniture"]
print(view.head(5).to_string(index=False))
```

```
23 customers who ordered in 2025
  customer  orders  revenue  storage  kitchen  industrial  furniture
Sharma Hardware    16   502775    0.656    0.310    0.000    0.034
Patel Kitchenware    7   174148    0.414    0.492    0.000    0.094
Green Leaf Hotels   17   331389    0.386    0.614    0.000    0.000
Coastal Foods      12   323631    0.481    0.000    0.519    0.000
Metro Mart         16   329298    0.726    0.274    0.000    0.000
```

The four share columns (named `storage_share`, `kitchen_share`, and so on in the file, shortened here to fit the page) give each customer's revenue split across Riverstone's four product categories. Sharma Hardware's split is 65.6% Storage, 31.0% Kitchen, 0% Industrial, and 3.4% Furniture. Check: $0.656 + 0.310 + 0.000 + 0.034 = 1.000$. ✓ That split is a 4-dimensional vector, and it's the one the next section uses to ask which customers *buy like* each other.

35.2 The dot product: a number for "how aligned"

Multiply and add

"Which customers buy the same mix of products as Sharma Hardware?" That question isn't about size. A small café and a large hotel chain can buy exactly the same mix. You need a measure of **direction**, not distance.

The tool is the **dot product**: multiply two vectors component by component, then add the results. Using product-mix vectors in the order [Storage, Kitchen, Industrial, Furniture]:

	Storage	Kitchen	Industrial	Furniture
Sharma Hardware	0.656	0.310	0.000	0.034
Metro Mart	0.726	0.274	0.000	0.000

	Storage	Kitchen	Industrial	Furniture
Coastal Foods	0.481	0.000	0.519	0.000

Sharma · Metro = $0.656 \times 0.726 + 0.310 \times 0.274 + 0.000 \times 0.000 + 0.034 \times 0.000 = 0.476256 + 0.084940 + 0 + 0 = \mathbf{0.561196}$

Sharma · Coastal = $0.656 \times 0.481 + 0.310 \times 0.000 + 0.000 \times 0.519 + 0.034 \times 0.000 = \mathbf{0.315536}$

The dot product is large when two vectors have big numbers **in the same places**. Sharma and Metro both put most of their money into Storage and Kitchen, so their products line up. Coastal Foods spends half its money on Industrial Crates, where Sharma spends nothing, so that half contributes zero.

Spreadsheet link. You have already met the dot product: it's `SUMPRODUCT`. With Sharma's shares in B2:E2 and Metro's in B3:E3, `=SUMPRODUCT(B2:E2, B3:E3)` returns 0.561196 in both Excel and Google Sheets. Chapter 4's weighted average discount was a dot product too.

Cosine similarity: removing size

A raw dot product still grows when vectors get longer. To measure direction alone, divide by both lengths. The result is the **cosine similarity**, the cosine of the angle between the two vectors:

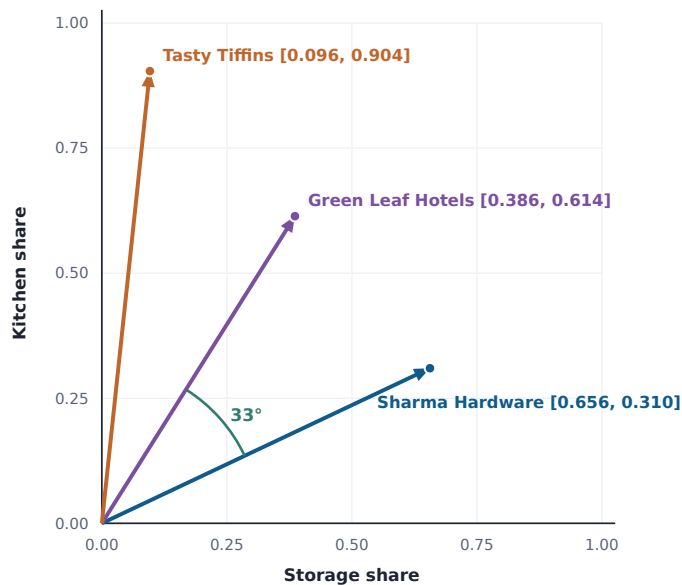
$$\text{cosine similarity} = (a \cdot b) \div (\text{length of } a \times \text{length of } b)$$

It runs from -1 (opposite directions) through 0 (at right angles, nothing in common) to 1 (pointing the same way). Revenue shares can't be negative, so here it runs from 0 to 1.

Work it for Sharma and Metro. Sharma's length is $\sqrt{(0.656^2 + 0.310^2 + 0^2 + 0.034^2)} = \sqrt{(0.430336 + 0.0961 + 0 + 0.001156)} = \sqrt{0.527592} = 0.72636$. Metro's is $\sqrt{(0.726^2 + 0.274^2)} = \sqrt{(0.527076 + 0.075076)} = \sqrt{0.602152} = 0.77598$. So:

$$\text{cosine(Sharma, Metro)} = 0.561196 \div (0.72636 \times 0.77598) = 0.561196 \div 0.563640 = \mathbf{0.9957}$$

For Sharma and Coastal, Coastal's length is $\sqrt{(0.481^2 + 0.519^2)} = 0.70762$, and the cosine is $0.315536 \div (0.72636 \times 0.70762) = \mathbf{0.6139}$. Sharma buys very much like Metro Mart and only partly like Coastal Foods.



Reading the picture

Each arrow is one customer's mix.
Its direction says what they buy;
its length doesn't matter for similarity.

Angle Sharma → Green Leaf: 33°
cosine in these 2 dimensions = 0.843

Angle Sharma → Tasty Tiffins: 59°
cosine in these 2 dimensions = 0.520

Smaller angle, higher cosine, more alike.

Industrial and Furniture shares are zero for these three customers, apart from Sharma's 3.4% Furniture.

Figure 35.1 – Product mixes as arrows (the Storage and Kitchen components only, two of the four). The smaller the angle between two arrows, the higher their cosine similarity, whatever their length.

Now let NumPy compare Sharma with all 23 customers at once:

```
share_cols = ["storage_share", "kitchen_share", "industrial_share", "furniture_share"]
mix = customers[share_cols].to_numpy()
names = customers["customer_name"].to_numpy()

sharma_mix = mix[0]
dots = mix @ sharma_mix
lengths = np.linalg.norm(mix, axis=1)
cosine = dots / (lengths * np.linalg.norm(sharma_mix))

ranking = pd.Series(cosine, index=names).sort_values(ascending=False)
print("Most similar mix to Sharma Hardware:")
print(ranking.head(5).round(4).to_string())
print("\nLeast similar:")
print(ranking.tail(3).round(4).to_string())
```

Most similar mix to Sharma Hardware:

Sharma Hardware	1.0000
Royal Banquets	0.9988
Evergreen Mart	0.9963
Metro Mart	0.9957
Om Sai Provisions	0.9873

Least similar:

Harbour Traders	0.5897
Tasty Tiffins	0.5198
Deccan Packaging	0.4676

How it works:

- `mix` is a 23×4 table of numbers: one row per customer, one column per category. `mix[0]` is the first row, Sharma Hardware.

- `@` is Python's operator for matrix multiplication. `mix @ sharma_mix` takes the dot product of **every row** with Sharma's vector, giving 23 numbers in one step. Section 35.3 explains why this works.
- `np.linalg.norm(mix, axis=1)` computes the length of each row (`axis=1` means "across the columns of each row").
- Dividing gives the 23 cosine similarities. Sharma's similarity with itself is 1, as it must be. ✓

Reading it. The customer whose mix most resembles Sharma Hardware's is not another hardware retailer but **Royal Banquets**, a hospitality customer in Lucknow. Segment labels and buying behavior are different things. The least similar are two wholesalers who spend heavily on Industrial Crates, and Tasty Tiffins, which buys almost nothing but kitchenware. This exact calculation, on real purchase histories, is the engine of the "customers who bought this also bought" recommender you build in Chapter 42, and dot products between word vectors are how the language models in Chapter 54 measure meaning.

35.3 Matrices: predicting every row at once

A dataset is a matrix

A **matrix** is a rectangular grid of numbers with rows and columns. Its **shape** is written rows × columns. `mix` above is a 23 × 4 matrix. In machine learning the feature matrix is traditionally called **X** (capital, because it's a matrix) and the thing you want to predict is **y** (lowercase, a vector with one value per row).

Matrix times vector

"If every order were worth the average, what revenue would each customer have brought in?" In 2025, Riverstone's 173 orders brought in ₹4,335,471, an average of ₹4,335,471 ÷ 173 = ₹25,060.53 per order. A very simple model says:

$$\text{predicted revenue} = ₹25,060.53 \times \text{orders} + ₹0$$

The two numbers the model uses, ₹25,060.53 and ₹0, are its **parameters** or **weights**: a slope w and an intercept b . To apply them to all 23 customers at once, write each customer's row as `[1, orders]`. The 1 is there so the intercept has something to multiply. Stack the rows into a matrix and put the weights in a vector `[b, w] = [0, 25,060.53]`. Then every prediction is the dot product of a row with the weight vector:

$$\text{Sharma: } [1, 16] \cdot [0, 25,060.53] = 1 \times 0 + 16 \times 25,060.53 = \text{₹400,968}$$

Matrix-times-vector means exactly this: take the dot product of each row of the matrix with the vector, and collect the answers. The number of columns in the matrix must equal the length of the vector (2 and 2 here), and the result has one number per row.

```

X = np.column_stack([np.ones(len(customers)), customers["orders"]])
y = customers["revenue"].to_numpy()
weights = np.array([0.0, 4335471 / 173]) # [intercept b, slope w]

predicted = X @ weights
print("X:", X.shape, " weights:", weights.shape, " predicted:", predicted.shape)
print("first three rows of X:\n", X[:3])
result = pd.DataFrame({"customer": names, "orders": customers["orders"],
                      "actual": y, "predicted": predicted.round(0).astype(int)})
result["error"] = result["predicted"] - result["actual"]
print(result.head(4).to_string(index=False))
print("total actual: ", y.sum())
print("total predicted:", round(predicted.sum()))

```

```

X: (23, 2) weights: (2,) predicted: (23,)
first three rows of X:
[[ 1. 16.]
 [ 1.  7.]
 [ 1. 17.]]

```

	customer	orders	actual	predicted	error
	Sharma Hardware	16	502775	400968	-101807
	Patel Kitchenware	7	174148	175424	1276
	Green Leaf Hotels	17	331389	426029	94640
	Coastal Foods	12	323631	300726	-22905
	total actual:		4335474		
	total predicted:		4335471		

How it works:

- `np.column_stack` glues columns side by side: a column of 23 ones and the column of orders, making the 23×2 matrix **X**.
- `X @ weights` multiplies a (23×2) matrix by a vector of length 2 and returns a vector of length 23: one prediction per customer.
- The **error** (prediction minus actual) is negative when the model under-predicts. Sharma Hardware is under-predicted by ₹101,807, because its orders are bigger than average.

Reconcile. The predictions add up to ₹4,335,471, exactly Riverstone's 2025 net revenue. ✓ That isn't luck: the slope is total revenue ÷ total orders, so multiplying back by total orders returns the total. (The `actual` column adds up to ₹4,335,474 because the practice file rounds each customer's revenue to the nearest rupee; three rupees of rounding across 23 customers.)

Spreadsheet link. `=MMULT(A2:B24, D2:D3)` multiplies a 23×2 range by a 2×1 range in both Excel and Google Sheets, returning 23 predictions (entered as a dynamic array in Microsoft 365 and Sheets). It's the same `@`.

Two more matrix words

- The **transpose** of a matrix, written **X^s** (`X.T` in NumPy), flips rows into columns. A 23×2 matrix becomes 2×23 . You'll see it in the gradient formula in section 35.5, and in PCA.
- **Mean-centering** subtracts each column's average from that column, so every column averages zero. PCA starts with it (section 35.10).

That's enough linear algebra for this part of the book. A neural network layer (Chapter 43) is this same $x @ weights$ with a bigger weight matrix, followed by a small twist.

35.4 Loss and the derivative: which way is downhill

A tiny dataset to learn on

The best way to understand how a model learns is to watch one learn from data small enough to check by hand. Take four Riverstone customers, with orders as the feature x and revenue in ₹ thousand as the target y :

Customer	Orders (x)	Revenue, ₹ thousand (y)
Om Sai Provisions	4	66
Patel Kitchenware	7	174
Evergreen Mart	8	198
Fresh Bowl Kitchens	12	297

(Revenue is rounded to the nearest thousand: Om Sai's ₹66,399 becomes 66. Small whole numbers keep the hand calculations readable.)

Start with the simplest possible model, a line through zero with one weight: **predicted revenue** = $w \times$ **orders**. The question is which w fits best.

The loss: one number for "how wrong"

For any w , you can measure how wrong the model is. The usual measure for predicting a number is the **mean squared error (MSE)**: for each row, take the error (prediction minus actual), square it, then average.

$$\text{loss}(w) = \text{average of } (w \times x - y)^2$$

Try $w = 20$ (₹20 thousand per order):

Customer	x	y	prediction $20x$	error	error ²
Om Sai	4	66	80	14	196
Patel	7	174	140	-34	1,156
Evergreen	8	198	160	-38	1,444
Fresh Bowl	12	297	240	-57	3,249

$$\text{loss}(20) = (196 + 1,156 + 1,444 + 3,249) \div 4 = 6,045 \div 4 = \mathbf{1,511.25}$$

A **loss function** turns "how good is this model?" into a single number where lower is better. Squaring does two jobs: it stops positive and negative errors cancelling out, and it punishes big misses much more than small ones (an error of 57 costs 3,249, more than an error of 14 does sixteen times over).

Training a model means finding the parameter values that make the loss as small as possible.

Everything else in this section is about how to find them without trying every value.

The derivative: the slope of the loss

Plot the loss for every w and you get a valley. You're standing at $w = 20$ and want to walk downhill. You need to know which direction is down, and how steep it is. That's what a **derivative** tells you: how much the loss changes for a tiny change in w . A negative derivative means increasing w lowers the loss; a positive one means increasing w raises it; zero means you're at the bottom (or a flat spot).

You can measure it without any calculus. Nudge w by a tiny amount and see what happens to the loss:

```
x = np.array([4, 7, 8, 12], dtype=float)
y = np.array([66, 174, 198, 297], dtype=float)

def loss(w):
    return np.mean((w * x - y) ** 2)

h = 0.001
print("loss(20)          =", round(loss(20), 4))
print("loss(20.001)      =", round(loss(20 + h), 4))
print("measured slope =", round((loss(20 + h) - loss(20)) / h, 2))
```

```
loss(20)          = 1511.25
loss(20.001)      = 1510.6651
measured slope = -584.93
```

Raising w by 0.001 lowered the loss by about 0.585, so the slope is about -585 per unit of w . Negative, so w should go **up**.

Calculus gives the exact slope as a formula. For mean squared error with one weight, the derivative is:

derivative of loss with respect to $w = 2 \times \text{average of (error} \times x)$

The error is multiplied by x because a weight's effect on a prediction is proportional to the feature it multiplies: nudging w moves Fresh Bowl's prediction (12 orders) three times as much as Om Sai's (4 orders). Check it at $w = 20$ using the error column above:

```
2 × (14 × 4 + (-34) × 7 + (-38) × 8 + (-57) × 12) ÷ 4 = 2 × (56 - 238 - 304 - 684) ÷ 4 = 2 × (-1,170) ÷ 4 = -585 ✓
```

The formula and the nudge agree. (The nudge gave -584.93 because 0.001 is small but not zero.)

Simplification note. You don't need to derive derivative formulas yourself. Libraries such as scikit-learn and PyTorch calculate them for you (PyTorch does it automatically for any model you write, which Chapter 43 shows). What you need is the meaning: *the derivative is the local slope of the loss, and its sign says which way to move.*

At the bottom of the valley the slope is zero. For this one-weight model you can solve for that point directly: $w = (\text{sum of } x \times y) \div (\text{sum of } x^2) = 6,630 \div 273 = \mathbf{24.29}$, about ₹24,290 of revenue per order.

35.5 Gradients: the slope in every direction

A real model has many parameters. Give the line an intercept, **predicted revenue** = $w \times \text{orders} + b$, and the loss now depends on two numbers. The loss surface is a bowl instead of a curve, and "downhill" has a direction on a map.

A **partial derivative** is the slope in one parameter's direction, holding the others still. The **gradient** is the vector of all the partial derivatives: one component per parameter. It points in the direction of steepest **increase** in the loss, so you walk the opposite way.

For mean squared error with w and b :

slope for $w = 2 \times \text{average of } (\text{error} \times x)$

slope for $b = 2 \times \text{average of } (\text{error})$

The intercept adds 1 to every prediction when it rises by 1, so its partial derivative has no x in it.

Work out the gradient at the start, $w = 0$ and $b = 0$. Every prediction is 0, so the errors are -66, -174, -198, and -297:

slope for $w = 2 \times (-66 \times 4 - 174 \times 7 - 198 \times 8 - 297 \times 12) \div 4 = 2 \times (-264 - 1,218 - 1,584 - 3,564) \div 4 = 2 \times (-6,630) \div 4 = \mathbf{-3,315}$

slope for $b = 2 \times (-66 - 174 - 198 - 297) \div 4 = 2 \times (-735) \div 4 = \mathbf{-367.5}$

The gradient is **[-3,315, -367.5]**. Both components are negative, so both w and b should increase. The w component is nine times bigger: the loss is far more sensitive to the slope than to the intercept, because w is multiplied by orders of 4 to 12 and b by 1. Remember that; it causes trouble in section 35.6.

Python link. With a feature matrix **X** that includes a column of ones, the whole gradient is one line: `2 * X.T @ (X @ weights - y) / len(y)`. `X @ weights - y` is the error vector, and `X.T @ errors` takes the dot product of each column (the ones, then the orders) with the errors. That's why the transpose appears in machine learning formulas.

35.6 Gradient descent, by hand and in NumPy

The update rule

Gradient descent repeats one move: compute the gradient, then step every parameter a small amount in the opposite direction.

$$\text{new parameter} = \text{old parameter} - \text{learning rate} \times \text{slope for that parameter}$$

The **learning rate** is the size of the step, a setting you choose. A setting chosen by you rather than learned from the data is a **hyperparameter**. Use a learning rate of **0.005**.

Three steps by hand

Step 1. Start at $w = 0$, $b = 0$. The gradient (section 35.5) is $[-3,315, -367.5]$, and the loss is $(66^2 + 174^2 + 198^2 + 297^2) \div 4 = (4,356 + 30,276 + 39,204 + 88,209) \div 4 = 162,045 \div 4 = \mathbf{40,511.25}$.

$$\text{new } w = 0 - 0.005 \times (-3,315) = \mathbf{16.575}$$

$$\text{new } b = 0 - 0.005 \times (-367.5) = \mathbf{1.8375}$$

Step 2. Predict with $w = 16.575$, $b = 1.8375$. Om Sai: $16.575 \times 4 + 1.8375 = 68.14$, an error of $+2.14$. Patel: $16.575 \times 7 + 1.8375 = 117.86$, error -56.14 . Evergreen: 134.44 , error -63.56 . Fresh Bowl: 200.74 , error -96.26 . The loss has fallen to **4,115.66**. The slopes are $2 \times (2.14 \times 4 - 56.14 \times 7 - 63.56 \times 8 - 96.26 \times 12) \div 4 = \mathbf{-1,024.03}$ for w and $2 \times (2.14 - 56.14 - 63.56 - 96.26) \div 4 = \mathbf{-106.91}$ for b (to two decimals; the code below carries full precision).

$$\text{new } w = 16.575 + 0.005 \times 1,024.03 = \mathbf{21.695}$$

$$\text{new } b = 1.8375 + 0.005 \times 106.91 = \mathbf{2.372}$$

Step 3. Predictions 89.15, 154.24, 175.93, 262.71; errors 23.15, -19.76 , -22.07 , -34.29 ; loss **647.26**; slopes -316.84 and -26.48 .

$$\text{new } w = 21.695 + 0.005 \times 316.84 = \mathbf{23.279}$$

$$\text{new } b = 2.372 + 0.005 \times 26.48 = \mathbf{2.504}$$

Step	w	b	Loss	Slope for w	Slope for b
1	0.000	0.000	40,511.25	-3,315.00	-367.50
2	16.575	1.838	4,115.66	-1,024.03	-106.91
3	21.695	2.372	647.26	-316.84	-26.48
after 3	23.279	2.504	316.36		

Three things to notice. The loss fell from 40,511 to 316 in three steps. The steps get **smaller on their own**, even though the learning rate is fixed, because the slope flattens as you approach the bottom. And *b* has barely moved.

The same three steps in NumPy

```
w, b = 0.0, 0.0
learning_rate = 0.005

for step in range(1, 4):
    error = (w * x + b) - y
    current_loss = np.mean(error ** 2)
    slope_w = 2 * np.mean(error * x)
    slope_b = 2 * np.mean(error)
    print(f"step {step}: w={w:7.3f} b={b:6.3f} loss={current_loss:10.2f} "
          f"slope_w={slope_w:9.2f} slope_b={slope_b:8.2f}")
    w = w - learning_rate * slope_w
    b = b - learning_rate * slope_b

print(f"after 3: w={w:7.3f} b={b:6.3f} loss={np.mean((w * x + b - y) ** 2):10.2f}")
```

```
step 1: w= 0.000 b= 0.000 loss= 40511.25 slope_w= -3315.00 slope_b= -367.50
step 2: w= 16.575 b= 1.838 loss= 4115.66 slope_w= -1024.03 slope_b= -106.91
step 3: w= 21.695 b= 2.372 loss= 647.26 slope_w= -316.84 slope_b= -26.48
after 3: w= 23.279 b= 2.504 loss= 316.36
```

How it works:

- Each pass through the loop does exactly what you did by hand: predict, find errors, compute the loss and both slopes, then update.
- `np.mean(error * x)` is "average of error $\times$ x", computed for all four customers at once.
- Printing *before* updating matches the hand table, where each row shows the parameters that produced that loss. Every number agrees with the table. ✓

Running it to the bottom

Let the same loop run for 1,000 steps, and compare with the exact best line. NumPy's `np.polyfit(x, y, 1)` finds the least-squares line directly, without gradient descent:

```
def descend(x_values, learning_rate, steps):
    w, b = 0.0, 0.0
    losses = []
    for _ in range(steps):
        error = (w * x_values + b) - y
        losses.append(np.mean(error ** 2))
        w -= learning_rate * 2 * np.mean(error * x_values)
        b -= learning_rate * 2 * np.mean(error)
    return w, b, losses

w, b, losses = descend(x, 0.005, 1000)
best_w, best_b = np.polyfit(x, y, 1)
best_loss = np.mean((best_w * x + best_b - y) ** 2)
print(f"gradient descent, 1000 steps: w={w:.2f} b={b:.2f} loss={losses[-1]:.2f}")
print(f"exact least squares: w={best_w:.2f} b={best_b:.2f} loss={best_loss:.2f}")
```

```
gradient descent, 1000 steps: w=27.12  b=-25.00  loss=109.48
exact least squares:          w=28.51  b=-37.21  loss=91.55
```

Reading it. After 1,000 steps the loss has fallen from 40,511 to 109.48, close to the best possible 91.55, but the parameters are further off than that suggests: w is 27.12 instead of 28.51, and b is -25.00 instead of -37.21 . The loss surface is a long, narrow valley. It's steep across the w direction and almost flat along the b direction, so the descent races down the steep wall and then creeps along the flat floor. A small loss doesn't guarantee the parameters have settled.

Watch out: a flat loss curve doesn't mean training has finished. If you judge convergence only by "the loss stopped changing much", you can stop with parameters that are still far from their best values. Here the intercept is off by ₹12,000. Check the parameters too, or better, fix the cause below.

The fix: scale the feature

The valley is narrow because orders (4 to 12) are much larger than the constant 1 that multiplies b . **Center** the feature by subtracting its average, 7.75 orders, and the two directions become equally steep. Now you can use a much bigger learning rate:

```
x_centered = x - x.mean()
w_c, b_c, losses_c = descend(x_centered, 0.05, 150)
print(f"centered, 150 steps: w={w_c:.2f}  b={b_c:.2f}  loss={losses_c[-1]:.2f}")
print(f"back on the original scale: b = {b_c - w_c * x.mean():.2f}")
```

```
centered, 150 steps: w=28.51  b=183.75  loss=91.55
back on the original scale: b = -37.21
```

With the feature centered, 150 steps land exactly on the least-squares answer: a slope of 28.51 and, converted back, an intercept of -37.21 . The intercept of the centered model, 183.75, has a plain meaning: the predicted revenue (₹183,750) for a customer with the average number of orders. About seven times fewer steps, and the right answer. This is why Chapter 36 treats scaling as a routine step before training.

What to tell the sales head. "Among these four customers, each extra order goes with about ₹28,500 of extra annual revenue." Don't read the intercept of -37.21 as "a customer with no orders brings in minus ₹37,210": no customer in the data has fewer than 4 orders, and a line is only trustworthy near the data it was fitted on. With four customers this is a teaching example, not a finding.

The learning rate

"How do I pick the learning rate?" Watch the loss. Run the original (uncentered) problem with three learning rates:

```
for lr in [0.0005, 0.005, 0.015]:
    _, _, run = descend(x, lr, 30)
    print(f"rate {lr:<6}: loss at steps 1-4 = "
          + ", ".join(f"{v:,.0f}" for v in run[:4])
          + f" ... step 30 = {run[-1]:,.0f}")
```

```
rate 0.0005: loss at steps 1-4 = 40,511, 35,141, 30,488, 26,456 ... step 30 = 913
rate 0.005 : loss at steps 1-4 = 40,511, 4,116, 647, 316 ... step 30 = 270
rate 0.015 : loss at steps 1-4 = 40,511, 46,679, 53,794, 61,999 ... step 30 = 2,521,040
```

- **0.0005 is too small.** The loss falls steadily but slowly: still 913 at step 30, more than three times the loss of the better rate.
- **0.005 is about right** for this problem. The loss falls from 40,511 to 316 in three steps and is at 270 by step 30.
- **0.015 is too large.** Each step overshoots the bottom of the valley and lands higher up the other side, so the loss **grows**. By step 30 it has passed 2.5 million, and it keeps climbing. A loss that shoots up to enormous numbers, or to `nan` ("not a number"), almost always means the learning rate is too high.

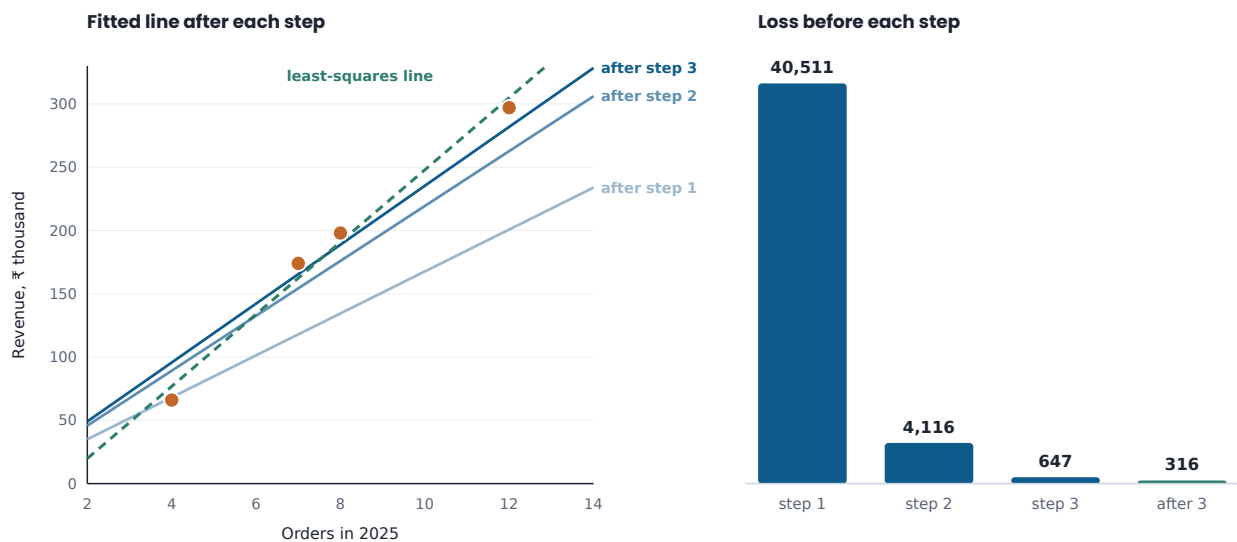


Figure 35.2 – Three steps of gradient descent by hand. Left: each step's line moves toward the least-squares line. Right: the loss before each step and after the third.

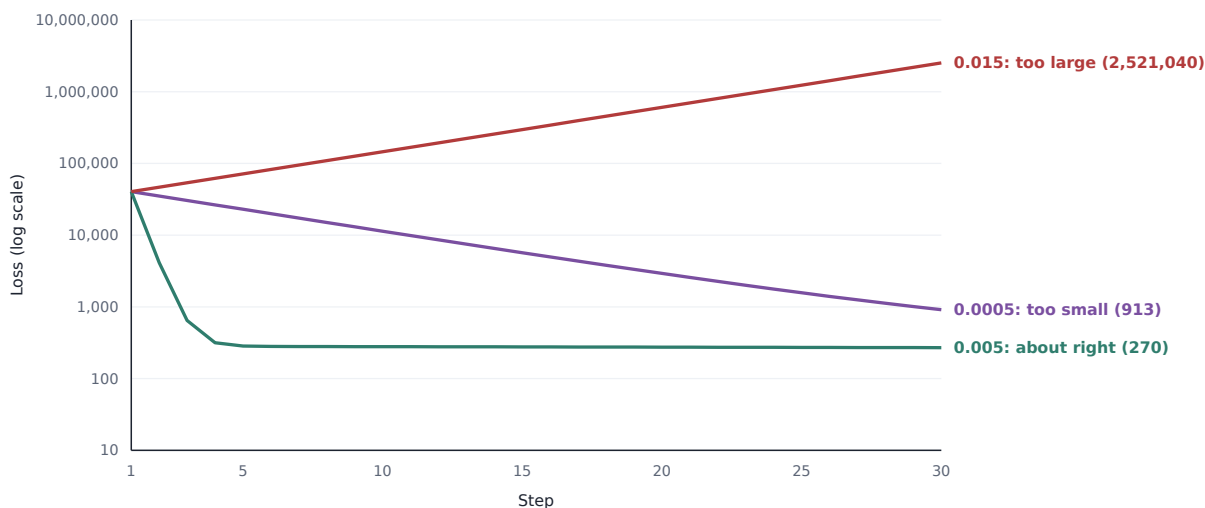


Figure 35.3 – The same problem with three learning rates. Too small crawls, about right drops fast, too large climbs away from the answer. The vertical axis is logarithmic: each gridline is ten times the one below.

Try it. In the learning-rate loop, try 0.014 and 0.0145. One of them converges, slowly, and one diverges, slowly. For this problem the dividing line is about 0.01447, which is 2 divided by the steepness of the valley's steepest direction. You don't need that formula at work; you need the habit of plotting the loss.

The pattern behind every model

You've now seen the three ingredients of almost every machine learning model:

1. **Parameters**, the numbers the model can adjust (w and b).
2. **A loss function** that scores how wrong the parameters are (mean squared error).
3. **An optimizer** that changes the parameters to lower the loss (gradient descent).

Linear regression (Chapter 37) uses exactly these, with more features. Logistic regression swaps in a different loss (section 35.9). A neural network (Chapter 43) has thousands or millions of parameters and computes its gradient automatically, but the loop is the one you wrote above. Real libraries add improvements such as **stochastic gradient descent** (computing each step on a random batch of rows instead of all of them, which is much faster on large data) and optimizers such as **Adam** that adjust the step size per parameter. They all follow the downhill slope.

35.7 Probability distributions: describing what's likely

A model that predicts a number with a straight line gives one answer. Many models instead give a **probability**: "this lead has a 20% chance of being won". To build and judge those, you need a way to describe how likely every possible outcome is. That description is a **probability distribution**. Chapter 21 introduces probability; this section adds the four distributions you'll meet most often in machine learning, each on real Riverstone data.

Bernoulli: one yes-or-no outcome

In 2025 Riverstone received **30** real sales enquiries (after removing duplicate web-form submissions, as in Chapter 13), and **6** were won. One lead is a single trial with two outcomes: won (1) or not won (0). A **Bernoulli distribution** describes it with one parameter, p , the probability of a 1.

$$P(\text{won}) = p \quad P(\text{not won}) = 1 - p$$

If $p = 0.2$, a lead is won with probability 0.2 and not won with probability 0.8. Every yes-or-no prediction in this book (will this customer churn, is this transaction fraud, will this lead convert) models the outcome as Bernoulli.

Binomial: how many yeses out of n

"If each lead has a 20% chance, how likely is it that exactly 6 of 30 are won?" Counting successes in n independent Bernoulli trials gives a **binomial distribution**:

$$P(\text{exactly } k \text{ won out of } n) = C(n, k) \times p^k \times (1 - p)^{(n - k)}$$

$C(n, k)$, read " n choose k ", counts how many different sets of k leads could be the winners. $C(30, 6) = 593,775$. So:

$$P(6 \text{ of } 30) = 593,775 \times 0.2^6 \times 0.8^{24} = 593,775 \times 0.000064 \times 0.0047224 = \mathbf{0.179}$$

Even when the true rate is exactly 20%, getting exactly 6 wins out of 30 happens only about 18% of the time. Five wins or seven are nearly as likely. Chapter 22 builds on this to ask whether a difference in win rates is real or noise.

Poisson: counts of events in a period

"How many orders should we expect in a week?" Riverstone's 173 orders in 2025 fell into 52 weeks, an average of $173 \div 52 = \mathbf{3.33}$ orders a week. When events arrive independently at a steady average rate, the number in a period follows a **Poisson distribution**, with one parameter, the average rate λ ("lambda"):

$$P(k \text{ events}) = \lambda^k \times e^{(-\lambda)} \div k!$$

Here e is the mathematical constant 2.71828, and $k!$ (" k factorial") is $1 \times 2 \times \dots \times k$. For zero orders in a week: $P(0) = 3.33^0 \times e^{(-3.33)} \div 0! = e^{(-3.33)} = 0.0359$. Over 52 weeks, that predicts $52 \times 0.0359 = \mathbf{1.9}$ weeks with no orders.

Does the model describe Riverstone's real weeks? Count them:

```
orders = pd.read_csv("orders_2025.csv", parse_dates=["order_date"])
week = (orders["order_date"].dt.dayofyear - 1) // 7 # week 0 = 1-7 January
counts = np.bincount(week, minlength=52)[:52]
rate = counts.mean()
print(f"{counts.sum()} orders in 52 weeks, average {rate:.2f} a week")

from math import exp, factorial
print("orders in a week | weeks observed | Poisson expects")
for k in range(9):
    expected = 52 * rate ** k * exp(-rate) / factorial(k)
    print(f"{k:>16} | {np.sum(counts == k):>14} | {expected:>15.1f}")
```

```
173 orders in 52 weeks, average 3.33 a week
orders in a week | weeks observed | Poisson expects
0 | 1 | 1.9
1 | 8 | 6.2
2 | 10 | 10.3
3 | 10 | 11.5
4 | 10 | 9.5
5 | 7 | 6.3
6 | 3 | 3.5
7 | 2 | 1.7
8 | 1 | 0.7
```

How it works: `dayofyear` numbers the days of 2025 from 1 to 365; subtracting 1 and dividing by 7 with `//` (whole-number division) gives a week number from 0. `np.bincount` counts how many orders fall into each week. The loop compares the number of weeks with k orders against $52 \times P(k)$.

Reading it. The Poisson model tracks the real weeks closely: it expects 10.3 weeks with two orders and 11.5 with three, and Riverstone had 10 of each. That's useful. A demand planner who knows orders are roughly Poisson with a rate of 3.3 a week knows that a week with 8 or more orders happens about once a year by chance alone, and doesn't need an explanation. (Riverstone's rate isn't really constant, since the festive season in October and November is busier; Chapter 40 handles seasonality properly.)

Normal: the bell curve, and when it lies

The **normal distribution** is the symmetric bell curve described by a mean and a standard deviation. Many methods assume it, often silently. Riverstone's order values have a mean of ₹25,060.53 and a standard deviation of ₹17,398.71. If order values were normal with those numbers, what would follow?

```
from statistics import NormalDist

values = orders["order_value"]
bell = NormalDist(mu=values.mean(), sigma=values.std())
print(f"mean ₹{values.mean():.2f} sd ₹{values.std():.2f} median ₹{values.median():.0f}")
above, below = 1 - bell.cdf(50000), bell.cdf(0)
print(f"above ₹50,000: normal predicts {above:.1%}, actual {np.mean(values > 50000):.1%}")
print(f"below ₹0: normal predicts {below:.1%}, actual {np.mean(values < 0):.1%}")
```

```
mean ₹25,060.53 sd ₹17,398.71 median ₹21,375
above ₹50,000: normal predicts 7.6%, actual 9.2%
below ₹0: normal predicts 7.5%, actual 0.0%
```

Reading it. The normal model says 7.5% of orders should have a **negative** value, which is impossible. It also underestimates large orders (7.6% predicted, 9.2% actual). Order values are **skewed**: most are moderate, a few are large, and none can be below zero, which is why the median (₹21,375) sits well below the mean. A normal distribution is a poor description of money amounts, counts of rare events, waiting times, and anything bounded at zero. Chapter 21 shows how to check the shape of data with a histogram before assuming.

Distribution	Describes	Parameters	Riverstone example	Used in
Bernoulli	one yes-or-no outcome	p	is this lead won?	logistic regression, classifiers (Ch 37)
Binomial	number of yeses in n tries	n, p	leads won out of 30	A/B tests (Ch 30), win-rate checks (Ch 22)
Poisson	count of events in a period	λ	orders per week	demand and anomaly models (Ch 40)
Normal	a measurement that clusters symmetrically	mean, standard deviation	<i>not</i> order values	linear regression errors (Ch 37), sampling (Ch 22)

35.8 Likelihood and maximum likelihood

Turning the question around

So far you've asked: *given* a parameter, how likely is the data? Machine learning asks the reverse: *given* the data, which parameter is most believable?

The **likelihood** of a parameter value is the probability of the data you actually observed, calculated as if that value were true. For Riverstone's 30 leads with 6 won, and a win rate p , the probability of that exact sequence of wins and losses is:

$$\text{likelihood}(p) = p^6 \times (1 - p)^{24}$$

(The $C(30, 6)$ from the binomial formula is the same for every p , so it doesn't change which p is best, and it's usually left out.)

Try three values:

p	$p^6 \times (1 - p)^{24}$
0.1	$0.000001 \times 0.079766 = 0.0000000798$
0.2	$0.000064 \times 0.0047224 = 0.000000302$
0.3	$0.000729 \times 0.00019158 = 0.000000140$

A win rate of 0.2 makes the observed data about 3.8 times as likely as 0.1 and about 2.2 times as likely as 0.3. The **maximum likelihood estimate (MLE)** is the parameter value that makes the observed data most likely.

Why everyone uses the log

Those likelihoods are tiny, and with 30,000 leads instead of 30 they'd be too small for a computer to store (they'd round to zero). So in practice you work with the **log-likelihood**: the natural logarithm of the likelihood. Logs turn multiplication into addition and keep numbers manageable, and because the log always rises when its input rises, the best p is the same.

$$\text{log-likelihood}(p) = 6 \times \ln(p) + 24 \times \ln(1 - p)$$

$$\text{at } p = 0.2: 6 \times \ln(0.2) + 24 \times \ln(0.8) = 6 \times (-1.60944) + 24 \times (-0.22314) = -9.65663 - 5.35545 = \mathbf{-15.0121}$$

Search every value of p from 0.001 to 0.999 and let NumPy find the best:

```
leads = pd.read_csv("leads_2025.csv")
won = leads["won"].to_numpy()
print(len(won), "leads,", won.sum(), "won")

p_grid = np.linspace(0.001, 0.999, 999)
log_likelihood = won.sum() * np.log(p_grid) + (len(won) - won.sum()) * np.log(1 - p_grid)
best = p_grid[np.argmax(log_likelihood)]
print(f"maximum likelihood estimate of p: {best:.3f}")
print(f"log-likelihood there: {log_likelihood.max():.4f}")
print(f"sample win rate: {won.mean():.3f}")
```

```
30 leads, 6 won
maximum likelihood estimate of p: 0.200
log-likelihood there: -15.0121
sample win rate: 0.200
```

How it works: `np.linspace(0.001, 0.999, 999)` makes 999 evenly spaced candidate values of p (0.001, 0.002, ..., 0.999). The log-likelihood is computed for all of them at once, and `np.argmax` returns the position of the largest.

The maximum likelihood estimate is **0.200**, exactly the sample win rate, $6 \div 30$. That's not a coincidence: for a Bernoulli outcome, the MLE is always the observed proportion, and for a Poisson count it's the observed average (3.33 orders a week). The MLE confirms the obvious in simple cases. Its value is that the same principle works for models where there's no obvious answer, like a lead-scoring model with twenty features.

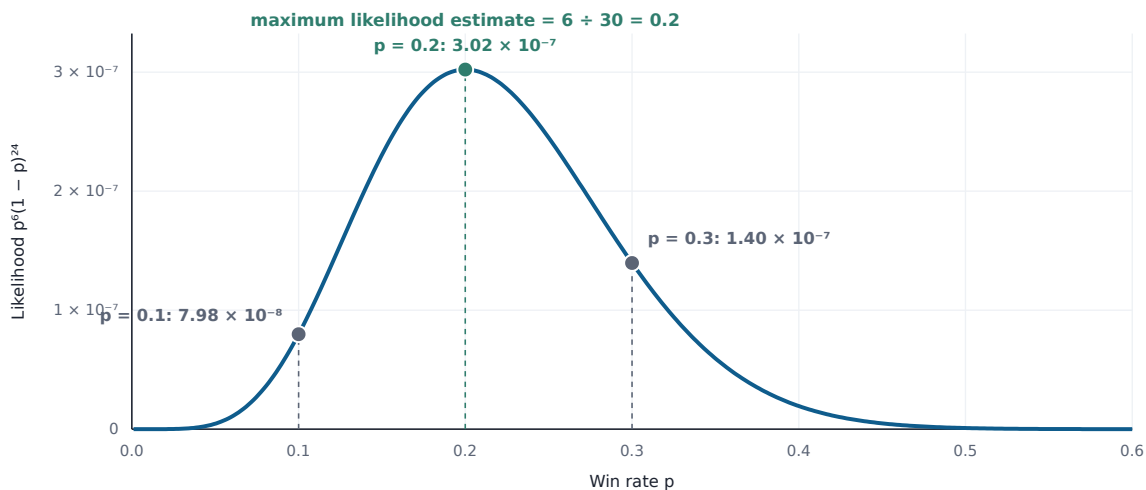


Figure 35.4 – The likelihood of each possible win rate, given 6 wins in 30 leads. The peak, the maximum likelihood estimate, is at 0.2. The curve is wide: rates from about 0.1 to 0.35 remain fairly plausible with only 30 leads.

Why this matters for models

Training a model is maximum likelihood estimation. A model's parameters decide the probabilities it assigns; fitting it means finding the parameters under which the training data is most likely. Two facts connect this to what you already know:

- For a model that predicts probabilities of yes-or-no outcomes, maximizing the log-likelihood is the same as minimizing **log loss** (section 35.9).
- For a model that predicts numbers, if you assume the errors are normally distributed, maximizing the likelihood is the same as minimizing **mean squared error**. That's where section 35.4's loss comes from.

35.9 Entropy, cross-entropy, and log loss

Entropy: how uncertain is an outcome?

"Before we look at anything about a lead, how uncertain are we about whether it will be won?"

Information theory measures uncertainty with **entropy**. For a yes-or-no outcome with probability p :

$$\text{entropy} = -(p \times \log_2 p + (1 - p) \times \log_2 (1 - p))$$

Measured with log base 2, the unit is the **bit**. A fair coin ($p = 0.5$) has entropy $-(0.5 \times (-1) + 0.5 \times (-1)) = 1$ **bit**, the most uncertain a yes-or-no outcome can be. Riverstone's lead outcome, at $p = 0.2$:

$$\begin{aligned} \text{entropy} &= -(0.2 \times \log_2 0.2 + 0.8 \times \log_2 0.8) = -(0.2 \times (-2.3219) + 0.8 \times (-0.3219)) = -(-0.4644 - 0.2575) \\ &= 0.722 \text{ bits} \end{aligned}$$

Less than 1 bit, because leads are usually not won, so you're less uncertain than with a coin. An outcome that's certain ($p = 0$ or 1) has entropy 0.

Information gain: the preview of decision trees

Entropy lets you measure how much a piece of information reduces uncertainty. Split Riverstone's leads by source:

Group	Leads	Won	Win rate	Entropy
All leads	30	6	20.0%	0.722
Website	14	1	7.1%	0.371
Every other source	16	5	31.3%	0.896

Check the website entropy: $-(1/14 \times \log_2(1/14) + 13/14 \times \log_2(13/14)) = -(0.0714 \times (-3.8074) + 0.9286 \times (-0.1069)) = 0.2720 + 0.0993 = 0.371$. ✓

After the split, the average uncertainty, weighted by group size, is $14/30 \times 0.371 + 16/30 \times 0.896 = 0.173 + 0.478 = \mathbf{0.651 \text{ bits}}$. Knowing whether a lead came through the website reduced uncertainty from 0.722 to 0.651 bits, an **information gain** of **0.071 bits**. A decision tree (Chapter 37) builds itself by trying every possible split and choosing the one with the biggest gain, then repeating inside each branch.

Cross-entropy and log loss: scoring probability predictions

Entropy measures uncertainty when you know the true probabilities. **Cross-entropy** measures how surprised you are, on average, when you use **predicted** probabilities and then see the real outcomes. For machine learning it's written with natural logs, averaged over rows, and called **log loss** (or **binary cross-entropy**):

$$\text{log loss} = -\text{average of } (y \times \ln(\text{predicted } p) + (1 - y) \times \ln(1 - \text{predicted } p))$$

For each row, only one term is active. If the lead was won ($y = 1$), the cost is $-\ln(\text{predicted } p)$. If not ($y = 0$), it's $-\ln(1 - \text{predicted } p)$.

Suppose a scoring tool gave five past leads these probabilities of being won, and here's what happened:

Lead	Predicted p	Won?	Cost
A	0.9	yes	$-\ln 0.9 = 0.105$
B	0.7	yes	$-\ln 0.7 = 0.357$
C	0.2	no	$-\ln 0.8 = 0.223$
D	0.1	no	$-\ln 0.9 = 0.105$
E	0.8	no	$-\ln 0.2 = 1.609$

$$\text{log loss} = (0.105 + 0.357 + 0.223 + 0.105 + 1.609) \div 5 = 2.399 \div 5 = \mathbf{0.480}$$

Look at lead E. The tool was 80% sure it would be won, and it wasn't. That one confident mistake costs more than the other four rows together. Log loss is harsh on confident wrong answers by design: a prediction of 0.99 for a lead that isn't won costs $-\ln 0.01 = 4.6$, and a prediction of exactly 1.0 costs infinity. A model trained on log loss learns to say "80%" only when it has earned it.

```
from sklearn.metrics import log_loss

y_true = np.array([1, 1, 0, 0, 0])
y_pred = np.array([0.9, 0.7, 0.2, 0.1, 0.8])
by_hand = -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))
print(f"log loss by hand:      {by_hand:.3f}")
print(f"log loss, sklearn:    {log_loss(y_true, y_pred):.3f}")

base_rate = np.full(len(won), won.mean()) # predict 0.2 for every lead
print(f"always predicting 0.2 on the 30 leads: {log_loss(won, base_rate):.4f} nats"
      f" = {log_loss(won, base_rate) / np.log(2):.4f} bits")
print(f"-(log-likelihood at p=0.2) / 30:      {-log_likelihood.max() / 30:.4f}")
```

```
log loss by hand:      0.480
log loss, sklearn:     0.480
always predicting 0.2 on the 30 leads: 0.5004 nats = 0.7219 bits
-(log-likelihood at p=0.2) / 30:      0.5004
```

How it works: `np.log` is the natural log, so the hand formula gives log loss in **nats** (the natural-log unit); dividing by $\ln 2$ converts nats to bits. `np.full(30, 0.2)` builds a "model" that predicts the base rate for every lead.

Reconcile three ideas. The three last numbers are the same quantity seen three ways:

- A model that always predicts the base rate of 0.2 has a log loss of **0.5004 nats**.
- In bits, that's **0.7219**, exactly the entropy of the outcome from earlier in this section. ✓ When your predictions match the true rate, cross-entropy equals entropy; any worse predictions give a higher cross-entropy.
- It's also the negative log-likelihood at $p = 0.2$ (section 35.8), $-(-15.0121) \div 30 = \mathbf{0.5004}$. ✓

Minimizing log loss is maximizing likelihood.

That gives you a benchmark for any lead score at Riverstone: a model that doesn't beat a log loss of about 0.50 on these leads knows nothing beyond the overall win rate. Chapter 39 uses the same idea to evaluate a real lead-scoring model.

Watch out: accuracy hides what log loss shows. A "model" that says *not won* for every lead is $24 \div 30 = \mathbf{80\% \text{ accurate}}$ on Riverstone's leads, while being useless. Accuracy counts right and wrong answers; log loss scores how well the probabilities match reality, and punishes false confidence. When someone quotes an accuracy figure, ask for the base rate first.

35.10 Principal component analysis (PCA)

The idea: one direction that says most

"We track seven numbers for every customer. Can we summarize them with fewer?" When several columns move together, they carry overlapping information. **Principal component analysis (PCA)** finds new axes, called **principal components**, that point along the directions where the data spreads out most. The first component captures as much of the spread as any single direction can; the second captures the most of what's left, at right angles to the first; and so on. Keep the first few and you've compressed the data with little loss.

"Spread" here is **variance**, the average squared distance from the mean (Chapter 21). PCA ranks directions by how much variance the data has along them.

A small example by hand

Five customers, two features: orders and revenue in ₹ ten-thousands (so Sharma Hardware's ₹502,775 becomes 50). Revenue is measured in ten-thousands so that the two columns have spreads of a similar size; the full example below shows what happens when they don't.

Customer	Orders	Revenue (₹10k)
Sharma Hardware	16	50
Metro Mart	16	33
Harbour Traders	10	41
Patel Kitchenware	7	17
Blue Bay Cafe	4	5

Step 1: center. The averages are $53 \div 5 = 10.6$ orders and $146 \div 5 = 29.2$. Subtract them:

Customer	Orders – 10.6	Revenue – 29.2
Sharma	5.4	20.8
Metro	5.4	3.8
Harbour	–0.6	11.8
Patel	–3.6	–12.2
Blue Bay	–6.6	–24.2

Step 2: the covariance matrix. The **covariance** of two columns is the average product of their centered values (dividing by $n - 1 = 4$, as for a sample standard deviation). It's positive when they rise together. The **covariance matrix** holds every column's variance on the diagonal and each pair's covariance off it.

variance of orders = $(5.4^2 + 5.4^2 + 0.6^2 + 3.6^2 + 6.6^2) \div 4 = (29.16 + 29.16 + 0.36 + 12.96 + 43.56) \div 4 = 115.2 \div 4 = \mathbf{28.8}$

variance of revenue = $(20.8^2 + 3.8^2 + 11.8^2 + 12.2^2 + 24.2^2) \div 4 = (432.64 + 14.44 + 139.24 + 148.84 + 585.64) \div 4 = 1,320.8 \div 4 = \mathbf{330.2}$

covariance = $(5.4 \times 20.8 + 5.4 \times 3.8 + (-0.6) \times 11.8 + (-3.6) \times (-12.2) + (-6.6) \times (-24.2)) \div 4 = (112.32 + 20.52 - 7.08 + 43.92 + 159.72) \div 4 = 329.4 \div 4 = \mathbf{82.35}$

So the covariance matrix is:

	Orders	Revenue
Orders	28.8	82.35
Revenue	82.35	330.2

Step 3: find the direction of most variance. The principal components are the **eigenvectors** of the covariance matrix, and the variance along each is its **eigenvalue**. In words: an eigenvector is a direction that the matrix only stretches, without turning, and the eigenvalue is how much it stretches. For a 2×2 covariance matrix with variances a and d and covariance c , the larger eigenvalue has a formula:

largest eigenvalue = $(a + d) \div 2 + \sqrt{((a - d) \div 2)^2 + c^2}$
 $= (28.8 + 330.2) \div 2 + \sqrt{((28.8 - 330.2) \div 2)^2 + 82.35^2} = 179.5 + \sqrt{(22,710.49 + 6,781.52)} = 179.5 + \sqrt{29,492.01} = 179.5 + 171.73 = \mathbf{351.23}$

The total variance is $28.8 + 330.2 = 359.0$, so the first component holds $351.23 \div 359.0 = \mathbf{97.8\%}$ of it. The second holds $359.0 - 351.23 = 7.77$, or 2.2%.

Its direction is found by solving $(28.8 - 351.23) \times u + 82.35 \times v = 0$, which gives $v = 3.915 \times u$. Scaled to length 1, that's **[0.247, 0.969]**: a direction that is mostly revenue, with some orders. It's a "customer size" axis.

Step 4: project. Each customer's **score** on the first component is the dot product of its centered row with the direction. Sharma: $5.4 \times 0.247 + 20.8 \times 0.969 = 1.34 + 20.15 = \mathbf{21.49}$. Blue Bay: $-6.6 \times 0.247 + (-24.2) \times 0.969 = -1.63 - 23.45 = \mathbf{-25.08}$. One number per customer now says most of what the two columns said.

```

small = np.array([[16, 50], [16, 33], [10, 41], [7, 17], [4, 5]], dtype=float)
centered = small - small.mean(axis=0)
cov = centered.T @ centered / (len(small) - 1)
print("covariance matrix:\n", cov)

eigenvalues, eigenvectors = np.linalg.eigh(cov)    # returned smallest first
order = np.argsort(eigenvalues)[::-1]
eigenvalues, eigenvectors = eigenvalues[order], eigenvectors[:, order]
pc1 = eigenvectors[:, 0] * np.sign(eigenvectors[1, 0])    # point it toward higher revenue
print("eigenvalues:", eigenvalues.round(2))
print("share of variance:", (eigenvalues / eigenvalues.sum()).round(3))
print("first component direction:", pc1.round(3))
print("scores on PC1:", (centered @ pc1).round(2))

```

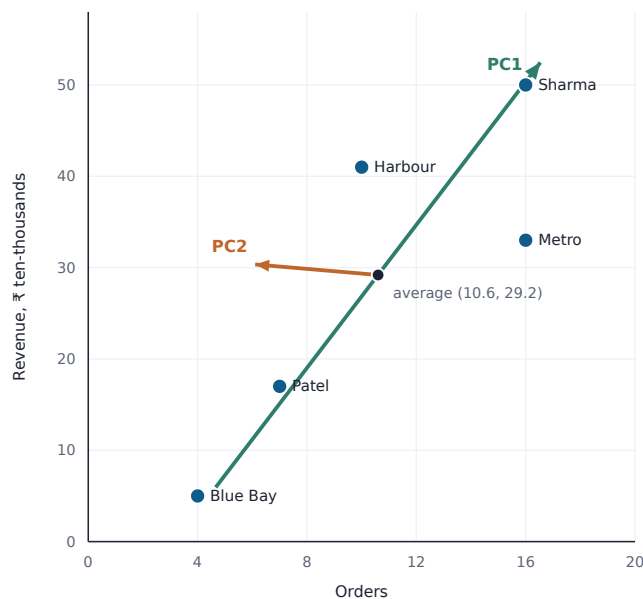
```

covariance matrix:
[[ 28.8  82.35]
 [ 82.35 330.2 ]]
eigenvalues: [351.23  7.77]
share of variance: [0.978 0.022]
first component direction: [0.247 0.969]
scores on PC1: [ 21.49  5.02 11.28 -12.71 -25.08]

```

How it works:

- `centered.T @ centered / 4` is the covariance matrix in one line: the dot product of each centered column with each other column, divided by $n - 1$. Compare it with the three hand calculations. ✓
- `np.linalg.eigh` finds eigenvalues and eigenvectors of a symmetric matrix (a covariance matrix is always symmetric). It returns them smallest first, so `argsort(...)[::-1]` reorders them largest first.
- An eigenvector's sign is arbitrary: `[0.247, 0.969]` and `[-0.247, -0.969]` describe the same axis. Multiplying by the sign of the revenue component makes "higher score = bigger customer" so the output is stable and readable.



Principal components

PC1 direction [0.247, 0.969]
variance 351.23 = 97.8% of total

PC2 direction [-0.969, 0.247]
variance 7.77 = 2.2% of total

The two components are at right angles.
 Orders are drawn 3× wider than revenue so the points are easier to see, which makes the right angle look slanted.

Figure 35.5 – The five customers and their principal components. The first component runs along the "customer size" direction and holds 97.8% of the variance; the short second component holds the rest.

All 23 customers, seven features

Now the real version: seven features per customer, which you can't draw. First, the mistake.

```
features = ["orders", "revenue", "avg_order_value", "avg_discount_pct",
            "storage_share", "kitchen_share", "industrial_share"]
F = customers[features].to_numpy(dtype=float)

def pca(matrix):
    centered = matrix - matrix.mean(axis=0)
    values, vectors = np.linalg.eigh(np.cov(centered, rowvar=False))
    order = np.argsort(values)::-1]
    return values[order] / values.sum(), vectors[:, order], centered @ vectors[:, order]

share_raw, vectors_raw, _ = pca(F)
print("UNSCALED share of variance, first 3 components:", share_raw[:3].round(4))
print("UNSCALED PC1 weights (size only):")
print(pd.Series(np.abs(vectors_raw[:, 0]), index=features).round(3).to_string())
```

```
UNSCALED share of variance, first 3 components: [0.9978 0.0022 0.    ]
UNSCALED PC1 weights (size only):
orders          0.000
revenue         1.000
avg_order_value 0.027
avg_discount_pct 0.000
storage_share   0.000
kitchen_share   0.000
industrial_share 0.000
```

Reading the wrong version. The first component "explains" 99.8% of the variance, which sounds wonderful and means nothing. Its weights are almost entirely on revenue. Revenue is measured in rupees and varies by hundreds of thousands, while shares vary between 0 and 1, so revenue's variance swamps everything. This unscaled PCA has rediscovered "revenue" and thrown away the other six features.

The fix is to **standardize** each column first: subtract the mean and divide by the standard deviation, so every feature has a standard deviation of 1 and an equal chance to matter.

```
F_std = (F - F.mean(axis=0)) / F.std(axis=0)
share, vectors, scores = pca(F_std)
signs = np.sign(vectors[features.index("industrial_share"), :])
signs[1] = np.sign(vectors[features.index("storage_share"), 1])
vectors = vectors * signs      # PC1 toward industrial buyers, PC2 toward storage
scores = F_std @ vectors
print("share of variance:", share.round(3))
print(f"first two components together: {share[:2].sum():.1%}")
loadings = pd.DataFrame(vectors[:, :2], index=features, columns=["PC1", "PC2"])
print(loadings.round(2).to_string())

from sklearn.decomposition import PCA
check = PCA().fit(F_std)
print("scikit-learn agrees:", np.allclose(check.explained_variance_ratio_, share))
```

```

share of variance: [0.535 0.205 0.154 0.084 0.02 0.002 0. ]
first two components together: 73.9%
           PC1  PC2
orders      0.30 -0.58
revenue     0.40 -0.47
avg_order_value 0.37 0.13
avg_discount_pct 0.44 0.15
storage_share 0.21 0.53
kitchen_share -0.45 -0.35
industrial_share 0.42 -0.06
scikit-learn agrees: True

```

How it works:

- `F.std(axis=0)` is each column's standard deviation; after dividing, every column has a spread of 1.
- `np.cov(centered, rowvar=False)` is the covariance matrix, with columns as variables.
- `np.allclose` checks that scikit-learn's `PCA` gives the same shares, allowing for tiny rounding differences. It does. (Its component signs can differ from ours, which is why the shares are compared, not the directions.)

Reading it. With every feature on the same scale, the first component holds 53.5% of the variance and the first two hold 73.9%. Read the component by its **weights** (also called **loadings**):

- **PC1** weighs heavily and positively on industrial share, average discount, revenue, and average order value, and negatively on kitchen share. It's a **"wholesale industrial buyer" axis**: big, discounted orders of crates at one end, kitchenware buyers at the other.
- **PC2** weighs positively on storage share and negatively on orders, revenue, and kitchen share. High scores mean **a few orders of storage products only**.

Plot every customer on these two axes, and the segments sort themselves, although PCA was never told which segment anyone belongs to:

```

plot_table = pd.DataFrame({"customer": names, "segment": customers["segment"],
                           "PC1": scores[:, 0].round(2), "PC2": scores[:, 1].round(2)})
print(plot_table.sort_values("PC1", ascending=False).head(5).to_string(index=False))
print(plot_table.groupby("segment")[["PC1", "PC2"]].mean().round(2).to_string())

```

```

      customer  segment  PC1  PC2
Harbour Traders Wholesale 3.75 -0.30
Coastal Foods Wholesale 3.04 -0.38
Deccan Packaging Wholesale 2.87 -0.51
Northgate Distributors Wholesale 2.76 -0.12
Western Logistics Wholesale 2.62 0.74
      PC1  PC2
segment
Hospitality -1.47 -0.48
Retail      -0.38 0.22
Wholesale   2.71 0.42

```

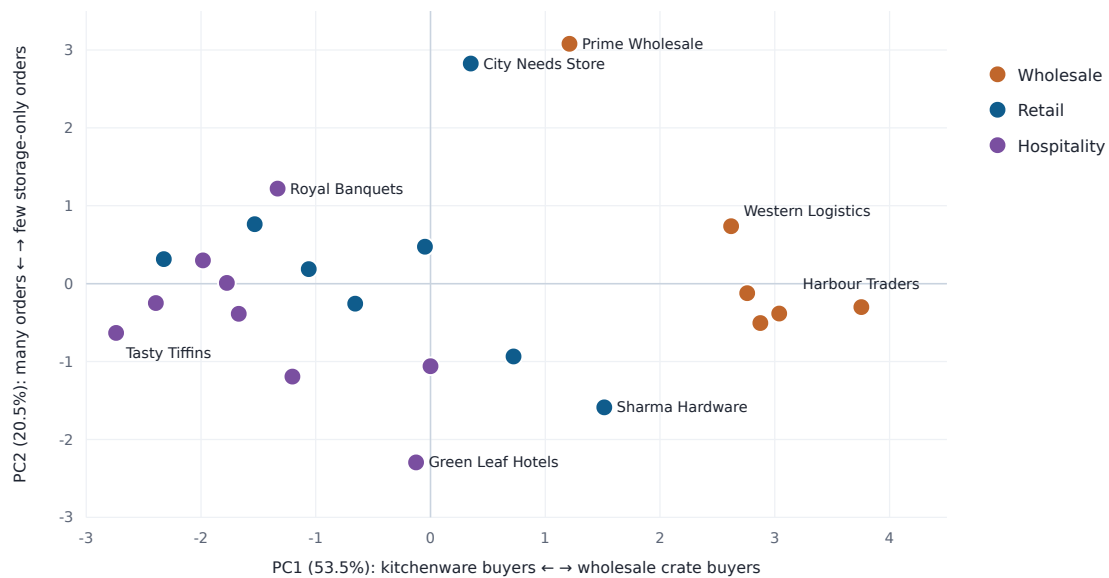


Figure 35.6 — All 23 customers on the first two principal components of seven standardized features. The five regular wholesale buyers sit together on the right; Prime Wholesale, a wholesaler with only two storage-only orders, sits apart, high on PC2.

What to tell the sales head. "Two combined measures describe about three-quarters of how our customers differ: how much they buy like a wholesale crate buyer, and whether they place a few storage-only orders. Prime Wholesale is labeled Wholesale but doesn't yet buy like one; it's worth a conversation." Chapter 38 uses PCA with clustering to build proper customer segments.

Watch out: PCA components are not features. "PC1" isn't a column anyone recorded; it's a weighted mix of all seven. It's useful for plotting, compressing, and feeding models many correlated columns, but it's harder to explain than the original features. If a manager needs to act on a result, show them the original columns behind it.

35.11 Which math sits under which model

Every idea in this chapter reappears. Use this table as a map for the rest of the book.

Math idea	What it does in a model	Where you'll see it next
Vectors and distance	Represent rows; find nearest neighbors	k-nearest neighbors (Ch 37), k-means and DBSCAN (Ch 38)
Dot product and cosine similarity	Measure similarity; weigh features	Linear models (Ch 37), recommenders (Ch 42), embeddings and attention (Ch 41, 54)
Matrix × vector	Predict every row at once	Linear and logistic regression (Ch 37), neural network layers (Ch 43)
Derivative and gradient	Say which way to change parameters	Every trained model; automatic in PyTorch (Ch 43)

Math idea	What it does in a model	Where you'll see it next
Gradient descent and learning rate	Train parameters step by step	Logistic regression, gradient boosting (Ch 37), neural networks (Ch 43, 53)
Scaling features	Make distances and gradients behave	Feature pipelines (Ch 36)
Bernoulli, binomial, Poisson, normal	Describe outcomes and counts	Classifiers (Ch 37), experiments (Ch 30), forecasting and anomalies (Ch 40)
Maximum likelihood	Choose the parameters that fit the data	Logistic regression, Naive Bayes (Ch 37), ARIMA (Ch 40)
Entropy and information gain	Pick the best split	Decision trees and random forests (Ch 37)
Cross-entropy / log loss	Train and score probability predictions	Logistic regression (Ch 37), calibration (Ch 39), neural networks (Ch 43), language models (Ch 54)
Covariance, eigenvectors, PCA	Compress correlated features	Dimensionality reduction (Ch 38), noise reduction before modeling

Interview extra point. When an interviewer asks "why log loss and not accuracy?" or "why not squared error for classification?", the answer that passes says log loss punishes confident mistakes. The answer that earns extra points adds the connection: *minimizing log loss is maximizing the likelihood of a Bernoulli model*, and then gives a benchmark, "a model that always predicts the base rate scores the entropy of the outcome, so I compare against that." The Machine Learning Question Bank (Chapter 74) has graded answers to this and other math-behind-the-model questions.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Measuring distance on features with different units	One feature (usually money) decides every distance; "nearest" customers are the ones with similar revenue	Standardize or scale features before any distance-based method
Using a raw dot product to compare direction	Big customers look similar to everyone	Divide by both lengths: use cosine similarity
Multiplying matrices with mismatched shapes	<code>ValueError: matmul: Input operand 1 has a mismatch in its core dimension</code>	Write the shapes down: (rows × columns) @ (columns) gives (rows)
Forgetting the column of ones	The fitted line is forced through zero	Add a column of ones for the intercept, or let the library fit it
Adding the gradient instead of subtracting it	The loss rises every step, smoothly	<code>new = old - learning_rate × slope</code>
Learning rate too high	Loss grows, jumps around, or becomes <code>nan</code> or <code>inf</code>	Divide the learning rate by 10 and plot the loss again

Mistake	Symptom	Fix
Learning rate too low	Loss falls, but very slowly; training "never finishes"	Multiply it by 3 to 10 while the loss still falls smoothly
Judging convergence by the loss alone	Loss looks flat while some parameters are still drifting	Scale features; check parameter changes as well as the loss
Assuming data is normal	Impossible predictions (negative order values), too few large values	Look at a histogram; use a distribution that fits the data's shape
Multiplying many small probabilities	Likelihood becomes 0.0 on a large dataset	Work with log-likelihood, a sum of logs
Judging a probability model by accuracy on imbalanced outcomes	An "80% accurate" model that never predicts a win	Compare log loss with the base-rate log loss; ask for the base rate
Predicting exactly 0 or 1	Log loss becomes infinite ($\inf$), or a library warns and clips the value	Keep probabilities strictly between 0 and 1
Running PCA without standardizing	First component "explains" 99%+ and loads on one big-unit column	Standardize every feature to standard deviation 1 first
Reading a PCA component's sign as meaningful	Two runs or two libraries give "opposite" components	Fix the sign by a rule (for example, a chosen feature positive) before interpreting
Treating principal components as business features	A manager can't act on "PC2 is high"	Explain results through the original columns behind the component

In the real world: Meera and the 91% lead score

It's the first week of February 2026. Anita Rao, Riverstone's Sales Head, forwards Meera Iyer a slide from a software vendor. The vendor's lead-scoring add-on for CRMs "predicts which leads will convert with **91% accuracy**", and the offer is ₹40,000 a month. Anita's note says: *"The sales team likes the demo. Is this worth it? I need an answer by Friday's review."*

Meera starts where this chapter taught her to start: **what would a model that knows nothing score?**

The base rate. From the CRM export behind Chapter 13, Riverstone had 30 real enquiries in 2025 and won 6, a win rate of 20%. A "model" that says *not won* for every lead is right 24 times out of 30: 80% accurate. The vendor's 91% is an improvement on that, but not the improvement it sounds like. And the vendor's figure comes from its other clients. Meera asks the vendor's account manager what share of leads those clients win. The answer is about 8%. At an 8% win rate, answering *not won* for every lead is 92% accurate, **better than the vendor's model**. The headline number, on its own, says nothing about whether the model has learned anything.

A measure that can't be gamed that way. Meera asks for probabilities instead of yes-or-no labels, and for **log loss** on held-out leads. Then she works out Riverstone's benchmark herself: a model that predicts 20% for every lead scores 0.50 on Riverstone's 2025 leads (section 35.9). Any score worth ₹40,000 a month has to beat 0.50 clearly on Riverstone's leads, not the vendor's clients'.

How much can 30 leads tell anyone? Meera looks back at the likelihood curve for the win rate (Figure 35.4). With 30 leads, win rates from about 10% to 35% are all reasonably believable. If a whole year of leads can't pin down the win rate more tightly than that, a test on a few weeks of leads can't tell a good score from a lucky one. She notes it plainly: the evaluation needs more leads than Riverstone gets in a month.

What's inside it. Finally, she asks the three questions every model can answer: *What are the parameters? What loss was minimized? What does it learn from?* The vendor explains it's a gradient-boosted model (Chapter 37) trained with log loss on activity data: email opens, website visits, and how quickly a rep responds. That last feature makes Meera pause. At Riverstone, reps respond fastest to leads they *already* think are promising, so a model can learn "fast response means likely win" from the reps' own judgment, and then look clever by repeating it. Chapter 36 calls this leakage.

Her reply to Anita, on Thursday evening:

"The 91% accuracy figure doesn't tell us much. At our 20% win rate, predicting 'no' for every lead is already 80% accurate, and at the vendor's clients' 8% win rate it's 92%. What matters is whether the probabilities beat a simple benchmark on our own leads. My recommendation: ask for a three-month pilot where the tool scores new leads but reps don't see the scores, so they can't influence the result. At the end, we compare its log loss with our benchmark of 0.50. If it isn't clearly better, we don't buy. I've also asked them to confirm the model doesn't use rep response time, which would make it look better than it is."

Anita takes the pilot proposal to the vendor. It is accepted, at no charge.

Notice what Meera didn't do. She didn't build a competing model or write any code. She used four ideas from this chapter (base rates, log loss, likelihood, and "parameters, loss, optimizer") to turn a sales pitch into a test with a pass mark. That's what the math under the models is for.

Tools

- **Python 3 with NumPy and pandas.** Every example ran on Python 3.12.3, NumPy 2.4.4, and pandas 3.0.2. The features used (`arrays`, `@`, `np.linalg.norm`, `np.linalg.eigh`, `np.polyfit`, `read_csv`) have been stable for many years, so older versions give the same results. Chapter 17 covers installation.
- **scikit-learn** (1.8.0 here) for two cross-checks: `sklearn.metrics.log_loss` and `sklearn.decomposition.PCA`. Chapter 36 introduces it properly.
- **Python's standard library:** `math (exp, factorial, comb)` and `statistics.NormalDist`. Nothing to install.
- **Excel and Google Sheets** have every calculation in this chapter: `SUMPRODUCT` (dot product), `MMULT` and `TRANSPOSE` (matrices), `BINOM.DIST`, `POISSON.DIST`, and `NORM.DIST` (distributions; Google Sheets also lists them under the older names `BINOMDIST` and `NORMDIST`), `LN` and `LOG` (logs, with `LOG(x, 2)` for bits), and `COVARIANCE.S` (covariance). Doing section 35.6's three steps in a spreadsheet, one row per step, is a good way to check your hand work.

- **Pen and paper.** The hand calculations are the point. Do them before running the code.
- **Visual explainers.** 3Blue1Brown's free *Essence of Linear Algebra* and *Essence of Calculus* video series animate vectors, matrices, and derivatives, and pair well with this chapter.
- **Companion files** (`companion/ch35/`): `make_ch35_data.py` (rebuilds the three CSV files from the seeded one-year generator, `companion/generate_riverstone_2025.py`), `customers_2025.csv`, `orders_2025.csv`, and `leads_2025.csv`. Run the chapter's code from that folder.

The project: gradient descent and PCA from scratch

Goal: a short notebook or script, written by you without machine learning libraries, that trains a model with gradient descent and compresses a table with PCA, and checks both against a library. It's a small, honest portfolio piece that shows you know what `.fit()` does.

Option A: your own data. Use a table from your work or life with at least 20 rows and 4 numeric columns (for example, monthly figures for a set of products or branches). Remove names and anything confidential first, and check you're allowed to use it.

Option B: Riverstone. Use `customers_2025.csv` and `orders_2025.csv` from the companion files.

Steps:

1. **Pick a prediction.** Choose one numeric target and one feature (Option B: predict each customer's revenue from their units bought). Write the question in one sentence.
2. **Hand-check first.** Take four rows and do two steps of gradient descent on paper, exactly as in section 35.6.
3. **Write the loop.** Implement gradient descent for w and b in NumPy, recording the loss at every step. Your first two steps must match your paper. ✓
4. **Tune the learning rate.** Try at least four learning rates, plot the loss curves on one chart, and write one sentence on each.
5. **Scale and compare.** Center (or standardize) the feature and train again. Report how many steps each version needs to get within 1% of the best loss, and compare your final w and b with `np.polyfit`.
6. **PCA by hand, then in code.** Pick 4 to 6 numeric columns. Standardize them, build the covariance matrix with $X.T @ X / (n - 1)$, and find eigenvalues and eigenvectors with `np.linalg.eigh`. Check the variance shares against scikit-learn's `PCA`.
7. **Interpret.** Name the first two components from their weights, plot the rows on them, and write three sentences a manager could understand.
8. **Write it up.** A one-page summary: the question, the two loss charts, the PCA plot, what you learned, and one thing that went wrong and how you spotted it.

Stretch goals:

- Extend your gradient descent to many features with the matrix form of the gradient, $2 * X.T @ (X @ weights - y) / n$, and train it on three features at once.

- Add **stochastic gradient descent**: compute each step on a random batch of 5 rows (with a fixed seed) and compare the loss curve with full-batch descent.
 - Implement logistic regression for `leads_2025.csv` using log loss and gradient descent, with one feature (whether the lead came from the website). Check that your predicted probabilities for the two groups approach $1 \div 14$ and $5 \div 16$.
-

You've got it when...

- ☐ I can compute a distance, a dot product, and a cosine similarity by hand, and explain why scaling changes distances.
- ☐ I can say what shape `x @ weights` returns before running it.
- ☐ I can explain in one sentence what a derivative tells me about a loss, and check a derivative formula by nudging a parameter.
- ☐ I can do a step of gradient descent on paper and get the same numbers as my code.
- ☐ Looking at a loss curve, I can tell a learning rate that's too small from one that's too large.
- ☐ I know that a flat loss curve doesn't prove the parameters have settled, and that scaling features often fixes slow training.
- ☐ I can name a distribution for yes-or-no outcomes, for counts per period, and for symmetric measurements, and I check the shape of data before assuming normal.
- ☐ I can explain maximum likelihood, and why minimizing log loss is the same thing for a classifier.
- ☐ I can calculate entropy, information gain, and log loss, and compare a model against the base-rate benchmark.
- ☐ I can run PCA by hand on two features and in NumPy on many, standardizing first, and describe a component from its weights.

When someone says "the model learns", and you can picture the parameters, the loss, and the downhill step, this chapter has done its job.

Recap

- A **vector** is an ordered list of numbers; each row of a dataset is one. **Euclidean distance** compares positions, and it depends on units, so **scale** features first.
- The **dot product** multiplies matching components and adds them. Divided by both **lengths**, it gives **cosine similarity**, which compares direction regardless of size.
- A **matrix** holds rows of vectors. **Matrix** $\times$ **vector** takes the dot product of each row with the weights, predicting every row at once. **Transpose** swaps rows and columns.
- A **loss function** such as **mean squared error** scores how wrong a model is. Training means finding **parameters** that minimize it.

- A **derivative** is the slope of the loss for one parameter; the **gradient** is the vector of slopes for all of them and points uphill.
- **Gradient descent** steps each parameter against its slope, scaled by the **learning rate** (a **hyperparameter**). Too small crawls; too large diverges; unscaled features make narrow valleys and slow training.
- **Bernoulli** and **binomial** describe yes-or-no outcomes, **Poisson** describes counts per period, and the **normal** distribution describes symmetric measurements, but not skewed amounts like order values.
- **Likelihood** scores parameter values by how probable they make the observed data. The **maximum likelihood estimate** is the best one; work with **log-likelihood** in practice.
- **Entropy** measures uncertainty in bits; **information gain** is how much a split reduces it, the rule decision trees use.
- **Cross-entropy**, or **log loss**, scores predicted probabilities and punishes confident mistakes. Minimizing it is maximizing likelihood, and a base-rate model's log loss is the benchmark to beat.
- **PCA** finds the directions of greatest **variance** using the **eigenvectors** of the **covariance matrix**. **Standardize** first, and read components by their weights.
- Almost every model is **parameters + a loss + an optimizer**.

Practice exercises

Exercises marked (*hand*) need only a calculator. Code exercises use the companion files in `companion/ch35/` and the variables defined in the chapter (`x`, `y`, `customers`, `leads`, and so on); the answers repeat any setup they need. Predict each result before you calculate it.

Warm-up

1. (*hand*) Green Leaf Hotels' product mix is $[0.386, 0.614, 0, 0]$ and Tasty Tiffins' is $[0.096, 0.904, 0, 0]$. Calculate their dot product, both lengths, and their cosine similarity. Are they more or less similar than Sharma Hardware and Coastal Foods (0.614)?
2. (*hand*) A feature matrix **X** has shape 150×6 . Which of these work, and what shape do they return: (a) `x @ w` with `w` of length 6; (b) `x @ w` with `w` of length 150; (c) `x.T @ x`; (d) `x @ x`?
3. (*hand*) For the four customers in section 35.4 and the model *predicted revenue* = $w \times \text{orders}$, calculate the loss and the derivative at $w = 25$. Should w go up or down?
4. (*hand*) What is the entropy, in bits, of a yes-or-no outcome with $p = 0.5$? With $p = 0.9$? Which is more uncertain, and why does that make sense?

Core

5. (*hand, then code*) Continue section 35.6's gradient descent for one more step, starting from $w = 23.279$ and $b = 2.504$ with a learning rate of 0.005. Calculate the errors, the loss, both slopes, and the new w and b . Check with code.

6. Write gradient descent for the one-weight model $\text{predicted revenue} = w \times \text{orders}$ (no intercept) on the four customers, starting at $w = 0$ with a learning rate of 0.005. Print w and the loss after 1, 2, 3, and 20 steps. Compare with the exact answer 24.29.
7. (hand) Riverstone averages 3.33 orders a week. Using the Poisson distribution, what's the probability of **at least one** order in a week? A new customer places orders at an average of 1.5 a month: what's the probability they place none in a given month?
8. Of the 14 website leads, 1 was won. Calculate the log-likelihood of the website leads' outcomes at $p = 1/14$ and at $p = 0.2$. Which value is the maximum likelihood estimate for website leads, and by how much is its log-likelihood higher?
9. Two scoring tools rate four leads, of which the first two were won. Tool A predicts [0.6, 0.4, 0.4, 0.4]; tool B predicts [0.95, 0.05, 0.05, 0.05]. Using a threshold of 0.5, what is each tool's accuracy? What is each tool's log loss? Which would you trust, and why?
10. Repeat section 35.10's standardized PCA on the customers **without** the `revenue` column (six features). What share of variance do the first component and the first two components hold now?

Stretch

- l1. For the four-customer problem with w and b (uncentered), run 50 steps of gradient descent for each learning rate in [0.002, 0.004, 0.008, 0.012, 0.014, 0.016]. Print the final loss for each. Which is the largest learning rate that still ends below the starting loss, and what happens a little above it?
- l2. For every customer, find the **other** customer with the most similar product mix (highest cosine similarity). Print the pairs for Metro Mart, Coastal Foods, and Tasty Tiffins, with their similarities.
- l3. Calculate the information gain of splitting Riverstone's 30 leads into "Referral or Cold call" versus every other source. Is it a better first split than "Website or not" (0.071 bits)?

Think about it (no calculation needed)

- l4. A different vendor claims its churn model is "95% accurate". Riverstone loses about 5% of its customers a year. What two questions would you ask before being impressed, and why?
- l5. A colleague runs k-means clustering (Chapter 38) on customers using `orders` and `revenue` in rupees. Every cluster seems to differ only by revenue. Explain what happened to the manager in two sentences, without the words *variance* or *standardize*.
- l6. A training run's loss is fine for 11 steps, then shows `nan` from step 12 on. Give two likely causes and what you'd try for each.

Key terms

vector · component · dimension · feature · Euclidean distance · length (norm) · element-by-element arithmetic · scaling · dot product · cosine similarity · matrix · shape · feature matrix (X) · target (y) · parameters (weights) · slope and intercept · matrix-times-vector (@) · transpose · mean-centering · loss function · mean squared error (MSE) · derivative · partial derivative · gradient · gradient descent · learning rate · hyperparameter · convergence · stochastic gradient descent · Adam · probability distribution · Bernoulli distribution · binomial distribution · Poisson distribution · normal distribution ·

skewed data · likelihood · maximum likelihood estimate (MLE) · log-likelihood · entropy · bit · information gain · cross-entropy · log loss (binary cross-entropy) · nat · base rate · principal component analysis (PCA) · principal component · variance · covariance · covariance matrix · eigenvector · eigenvalue · standardization · loadings · explained variance

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 36, The Machine Learning Workflow & Feature Engineering**, turns this chapter's scaling lesson into a standard pipeline step, and builds the leakage-free lead-scoring features that Meera worried about.
- **Chapter 37, Supervised Learning Algorithms**, uses every idea here: least squares and gradient descent in linear regression, log loss in logistic regression, likelihood in Naive Bayes, entropy in decision trees, and distance in k-nearest neighbors.
- **Chapter 38, Unsupervised Learning**, applies distance and PCA to cluster Riverstone's customers, and adds t-SNE and UMAP.
- **Chapter 39, Evaluation, Tuning, Interpretation & Honesty**, evaluates a real lead-scoring model with log loss, calibration, and cost-based thresholds, starting from the base-rate benchmark.
- **Chapter 43, A First Look at Deep Learning**, computes a neuron by hand and trains a network in PyTorch with the same loop as section 35.6, with gradients calculated automatically.
- **Chapter 54, Generative AI & Large Language Models**, shows dot products between vectors at work inside embeddings and attention.
- **Interview preparation:** the Machine Learning Question Bank (Chapter 74) covers gradient descent, learning rates, log loss, bias and variance, and PCA questions with graded answers; the Statistics, Probability & Experimentation Bank (Chapter 73) covers distributions and likelihood.

Answers to practice exercises

(In the finished book these move to Appendix G. Every calculation was checked, and every code output shown is real.)

1. Dot product = $0.386 \times 0.096 + 0.614 \times 0.904 = 0.037056 + 0.555056 = \mathbf{0.592112}$. Lengths: $\sqrt{(0.386^2 + 0.614^2)} = \sqrt{(0.148996 + 0.376996)} = \sqrt{0.525992} = 0.72525$; $\sqrt{(0.096^2 + 0.904^2)} = \sqrt{(0.009216 + 0.817216)} = \sqrt{0.826432} = 0.90908$. Cosine = $0.592112 \div (0.72525 \times 0.90908) = 0.592112 \div 0.659310 = \mathbf{0.898}$. They're **more** similar than Sharma and Coastal (0.614): both are kitchen-heavy, though Tasty Tiffins much more so. Check:

```
green_leaf = np.array([0.386, 0.614, 0, 0])
tasty = np.array([0.096, 0.904, 0, 0])
print(round(green_leaf @ tasty, 6),
      round(green_leaf @ tasty / (np.linalg.norm(green_leaf) * np.linalg.norm(tasty)), 3))
```

0.592112 0.898

2. (a) Works: $(150 \times 6) @ (6)$ returns a vector of **150** predictions, one per row. (b) Fails: the matrix has 6 columns, so the vector must have 6 numbers. (c) Works: $(6 \times 150) @ (150 \times 6)$ returns a **6 × 6** matrix, the shape of a covariance matrix. (d) Fails: 6 columns can't meet 150 rows. The rule: the inner numbers must match, and the outer numbers give the result's shape.

3. Predictions $25 \times [4, 7, 8, 12] = [100, 175, 200, 300]$; errors $[34, 1, 2, 3]$; squares $[1,156, 1, 4, 9]$.

$\text{loss}(25) = 1,170 \div 4 = \mathbf{292.5}$

$\text{derivative} = 2 \times (34 \times 4 + 1 \times 7 + 2 \times 8 + 3 \times 12) \div 4 = 2 \times (136 + 7 + 16 + 36) \div 4 = 2 \times 195 \div 4 = \mathbf{97.5}$

The derivative is positive, so increasing w would raise the loss: w should go **down**, toward the best value of 24.29. Notice that Om Sai's single large error (34) dominates both numbers.

4. At $p = 0.5$: $-(0.5 \times \log_2 0.5 + 0.5 \times \log_2 0.5) = -(-0.5 - 0.5) = \mathbf{1 \text{ bit}}$. At $p = 0.9$: $-(0.9 \times \log_2 0.9 + 0.1 \times \log_2 0.1) = -(0.9 \times (-0.1520) + 0.1 \times (-3.3219)) = 0.1368 + 0.3322 = \mathbf{0.469 \text{ bits}}$. The 50–50 outcome is more uncertain: you have no idea which way it will go. At 90%, you'd usually guess right, so there's less uncertainty to remove.

5. Predictions: $23.279 \times 4 + 2.504 = 95.62$; $\times 7 + 2.504 = 165.46$; $\times 8 + 2.504 = 188.74$; $\times 12 + 2.504 = 281.85$. Errors: 29.62, -8.54, -9.26, -15.15. Loss = $(877.3 + 72.9 + 85.7 + 229.5) \div 4 = \mathbf{316.4}$, matching the "after 3" row. ✓ Slope for $w = 2 \times (29.62 \times 4 - 8.54 \times 7 - 9.26 \times 8 - 15.15 \times 12) \div 4 = 2 \times (118.48 - 59.78 - 74.08 - 181.80) \div 4 = \mathbf{-98.6}$. Slope for $b = 2 \times (29.62 - 8.54 - 9.26 - 15.15) \div 4 = \mathbf{-1.67}$. New $w = 23.279 + 0.005 \times 98.6 = \mathbf{23.772}$; new $b = 2.504 + 0.005 \times 1.67 = \mathbf{2.512}$. The code carries full precision, so its last digits differ slightly from the rounded hand values (-15.14, -98.55, -1.66, 2.513):

```
w, b = 0.0, 0.0
for step in range(4):
    error = (w * x + b) - y
    slope_w, slope_b = 2 * np.mean(error * x), 2 * np.mean(error)
    if step == 3:
        print("errors:", error.round(2), " loss:", round(np.mean(error ** 2), 2))
        print(f"slope_w={slope_w:.2f} slope_b={slope_b:.2f}")
    w, b = w - 0.005 * slope_w, b - 0.005 * slope_b
print(f"after step 4: w={w:.3f} b={b:.3f}")
```

```
errors: [ 29.62 -8.54 -9.26 -15.14] loss: 316.36
slope_w=-98.55 slope_b=-1.66
after step 4: w=23.772 b=2.513
```

The slope for b is now tiny (-1.67), so b barely moves: the narrow valley from section 35.6, in numbers.

6.

```
w = 0.0
for step in range(1, 21):
    slope = 2 * np.mean((w * x - y) * x)
    w = w - 0.005 * slope
    if step in (1, 2, 3, 20):
        print(f"after step {step:>2}: w={w:.3f}  loss={np.mean((w * x - y) ** 2):.2f}")
print(f"exact: w={np.sum(x * y) / np.sum(x * x):.3f}")
```

```
after step  1: w=16.575  loss=4315.49
after step  2: w=21.838  loss=666.73
after step  3: w=23.508  loss=298.91
after step 20: w=24.286  loss=257.68
exact: w=24.286
```

With a single weight there's no narrow valley, so descent reaches the exact answer, 24.286, well within 20 steps. The loss at the best one-weight line (257.68) is higher than the two-parameter best (91.55): the intercept helps this model fit.

7. $P(\text{at least one}) = 1 - P(0) = 1 - e^{(-3.33)} = 1 - 0.0359 = \mathbf{0.964}$, or 96.4%. For the new customer, $P(0 \text{ in a month}) = e^{(-1.5)} = \mathbf{0.223}$: they'd go a whole month without ordering in about one month out of four and a half, **with no change in behavior**. A "quiet month" alert for such a customer would fire often by chance alone.

8.

```
website = leads.loc[leads["source"] == "Website", "won"].to_numpy()
wins, n = website.sum(), len(website)

def bernoulli_log_likelihood(p):
    return wins * np.log(p) + (n - wins) * np.log(1 - p)

print(f"{n} website leads, {wins} won")
print(f"log-likelihood at p=1/14: {bernoulli_log_likelihood(1 / 14):.4f}")
print(f"log-likelihood at p=0.2: {bernoulli_log_likelihood(0.2):.4f}")
print(f"difference: {bernoulli_log_likelihood(1 / 14) - bernoulli_log_likelihood(0.2):.4f}")
```

```
14 website leads, 1 won
log-likelihood at p=1/14: -3.6025
log-likelihood at p=0.2: -4.5103
difference: 0.9078
```

The MLE for website leads is their own win rate, $1/14 = \mathbf{0.071}$, and its log-likelihood is about 0.91 higher. That's a likelihood ratio of $e^{0.91}$, about 2.5: the website leads' outcomes are 2.5 times as likely under 7.1% as under 20%. With only 14 leads that's suggestive, not conclusive; Chapter 22 shows how to test it.

9.

```
outcome = np.array([1, 1, 0, 0])
tool_a = np.array([0.6, 0.4, 0.4, 0.4])
tool_b = np.array([0.95, 0.05, 0.05, 0.05])
for name, pred in [("A", tool_a), ("B", tool_b)]:
    accuracy = np.mean((pred >= 0.5) == outcome)
    print(f"tool {name}: accuracy {accuracy:.0%}, log loss {log_loss(outcome, pred):.3f}")
```

```
tool A: accuracy 75%, log loss 0.612
tool B: accuracy 75%, log loss 0.787
```

Both tools are **75%** accurate: each gets the second lead wrong. But tool A's log loss (0.612) is lower than tool B's (0.787). Tool B was 95% sure the second lead would be lost, and it was won; that one confident error costs $-\ln 0.05 = 3.00$, more than all of tool A's costs together. Trust **tool A**: its probabilities are more honest. Accuracy can't see the difference; log loss can. (With four leads, this is only an illustration: Chapter 39 evaluates on hundreds.)

10.

```
features_6 = [f for f in features if f != "revenue"]
F6 = customers[features_6].to_numpy(dtype=float)
share_6, _, _ = pca((F6 - F6.mean(axis=0)) / F6.std(axis=0))
print("share of variance:", share_6.round(3))
print(f"PC1: {share_6[0]:.1%}    PC1 + PC2: {share_6[:2].sum():.1%}")
```

```
share of variance: [0.542 0.199 0.144 0.092 0.022 0.    ]
PC1: 54.2%    PC1 + PC2: 74.1%
```

Almost nothing changes: the first component holds 54.2% instead of 53.5%, and the first two hold 74.1% instead of 73.9%. Revenue is largely orders $\times$ average order value, so the other columns already carried most of what it said. When dropping a column barely changes a PCA, that column was mostly repeating the others.

11.

```
for lr in [0.002, 0.004, 0.008, 0.012, 0.014, 0.016]:
    _, _, run = descend(x, lr, 50)
    print(f"learning rate {lr:<5}: loss after 50 steps = {run[-1]:>14,.2f}")
```

```
learning rate 0.002: loss after 50 steps =      274.23
learning rate 0.004: loss after 50 steps =      265.94
learning rate 0.008: loss after 50 steps =      250.46
learning rate 0.012: loss after 50 steps =      236.33
learning rate 0.014: loss after 50 steps =      289.34
learning rate 0.016: loss after 50 steps = 6,239,746,262,063.38
```

Up to **0.014**, the loss ends far below its starting value of 40,511. At 0.014 it converges only slowly, because each step overshoots and swings back a little less each time. At 0.016 each overshoot is bigger than the last, and the loss explodes. The boundary is about 0.01447 (section 35.6's Try it), and it's sharp: a learning rate a few percent too high doesn't train slowly, it fails completely. That's why the safe habit is to start low and increase.

12.

```
lengths = np.linalg.norm(mix, axis=1)
similarity = (mix @ mix.T) / np.outer(lengths, lengths)
np.fill_diagonal(similarity, -1) # ignore each customer's match with itself
for who in ["Metro Mart", "Coastal Foods", "Tasty Tiffins"]:
    i = list(names).index(who)
    j = np.argmax(similarity[i])
    print(f"{who:<14} -> {names[j]:<23} {similarity[i, j]:.4f}")
```

```
Metro Mart      -> Evergreen Mart      1.0000
Coastal Foods   -> Harbour Traders     0.9994
Tasty Tiffins   -> Sea Breeze Hotel     0.9939
```

How it works: `mix @ mix.T` takes the dot product of every customer with every other customer in one step, a 23×23 matrix. `np.outer(lengths, lengths)` builds the matching 23×23 table of length products, so dividing gives all cosine similarities at once. Setting the diagonal to -1 stops a customer matching itself. This all-pairs similarity matrix is exactly what Chapter 42's item-to-item recommender computes, with products instead of customers.

13.

```
def entropy(p):
    return 0.0 if p in (0, 1) else -(p * np.log2(p) + (1 - p) * np.log2(1 - p))

in_group = leads["source"].isin(["Referral", "Cold call"])
parent = entropy(leads["won"].mean())
children = sum(len(g) / len(leads) * entropy(g["won"].mean())
               for _, g in leads.groupby(in_group))
print(f"group sizes: {in_group.sum()} referral or cold call, {(~in_group).sum()} other")
print(f"won: {leads.loc[in_group, 'won'].sum()} and {leads.loc[~in_group, 'won'].sum()}")
print(f"information gain: {parent - children:.3f} bits")
```

```
group sizes: 8 referral or cold call, 22 other
won: 4 and 2
information gain: 0.133 bits
```

Referral and cold-call leads won 4 of 8 (50%); every other source won 2 of 22 (9.1%). The information gain is larger than the website split's 0.071 bits, so a decision tree would choose this split first. With 30 leads, though, one or two different outcomes would change the ranking; Chapter 37 shows how trees guard against splitting on noise.

14. First: *what does a model that predicts "no churn" for everyone score?* At a 5% churn rate, it's 95% accurate, the same as the vendor's model, while catching nobody. Second: *how well does it rank and predict probabilities on our data?* Ask for log loss (compared with the base-rate benchmark) and for how many of the customers who actually churned it flagged (recall, Chapter 39), measured on customers the model wasn't trained on. The common wrong reaction is to be impressed because 95% is close to 100%.

15. "The method groups customers by how far apart they are, and a difference of ₹1 counted exactly as much as a difference of one order. Since revenue differences run into lakhs and order differences into single digits, revenue decided everything, and the order counts were effectively ignored." (The fix is to put both on the same scale first, section 35.10.)

16. Most likely, **the learning rate is too high**: the loss grows each step until the numbers overflow to `inf` and then `nan`. Try dividing the learning rate by 10, and plot the loss for the first 12 steps to see whether it was already rising. Second, **a value that breaks a formula**: a missing value in the data, a division by zero, or a predicted probability of exactly 0 or 1 inside a log. Check the inputs for missing values, and check the batch or row being processed at step 12. (A third cause, features on very different scales, makes the first cause more likely; scaling often fixes both.)